



Incremental Graph Construction Enables Robust Spectral Clustering of Texts

Marko Pranjic¹ · Boshko Koloski¹ · Nada Lavrač^{1,2} · Senja Pollak¹ · Marko Robnik-Šikonja²

Received: 3 March 2026 / Revised: 28 July 2026 / Accepted: 25 August 2026
© The Author(s) 2026

Abstract

Neighborhood graphs over document embeddings are a common component of text-mining pipelines, supporting topic discovery, deduplication, semi-supervised label propagation, and retrieval-index construction, and they are a critical but fragile step in spectral clustering of text embeddings. On realistic text datasets, standard k -NN graphs can contain many disconnected components at practical sparsity levels (small k), making spectral clustering degenerate and sensitive to hyperparameters. We introduce an incremental k -NN graph construction algorithm in which each new node is linked to its k nearest previously inserted nodes; this guarantees a connected graph for any k . We provide an inductive proof of connectedness and discuss implications for incremental updates when new documents arrive. We validate the approach on spectral clustering of SentenceTransformer embeddings using Laplacian eigenmaps across eleven sentence- and paragraph-level clustering tasks drawn from six dataset sources in the Massive Text Embedding Benchmark. Compared to standard k -NN graphs, our method outperforms in the low- k regime where disconnected components are prevalent, and matches standard k -NN at larger k . The advantage at low k persists when the standard k -NN graph is repaired with a minimum spanning tree, while our construction avoids the dense distance matrix an exact repair requires and is consequently far cheaper to build, in both time and memory. The code is available on https://github.com/bkoloski/incremental_clustering_graphs.

Keywords Spectral clustering · Incremental graph construction · k -nearest neighbor (k -nn) graphs · Text embeddings · Laplacian eigenmaps

Marko Pranjic and Boshko Koloski have contributed equally to this work.

Editors: Gianvito Pio, Jurica Levatić, Nikola Simidjievski.

✉ Boshko Koloski
boshko.koloski@ijs.si

¹ Jožef Stefan Institute and International Postgraduate School, Ljubljana, Slovenia

² Faculty of Computer and Information Science, University of Ljubljana, Ljubljana, Slovenia

1 Introduction

Graph-based machine learning has achieved strong results in domains with inherent relational structure, including protein interactions (Jha et al., 2022), materials science (Reiser et al., 2022), and travel-time estimation (Derrow-Pinion et al., 2021). Recently, graph methods have been extended to data without explicit graph structure, such as tabular data (Margeloiu et al., 2024), text (Koloski et al., 2023), and medical imaging (Zaripova et al., 2023), by constructing a neighborhood graph in which data points become nodes and edges encode pairwise similarity. For text in particular, such document-similarity graphs support a range of practical tasks: unsupervised topic and theme discovery in large corpora, near-duplicate and plagiarism detection, semi-supervised label propagation from a few annotated documents, and the construction of retrieval indices, including those used in retrieval-augmented generation. In all of these tasks, the connectivity of the graph determines reachability: an isolated component of documents cannot receive a propagated label, be retrieved through the index, or be merged with a related theme. The fragility studied in this paper therefore has direct practical consequences.

The two most common construction methods are ϵ -threshold graphs and k -nearest neighbor graphs (k -NN). Both rely on a distance metric: the former connects all pairs within distance ϵ , and the latter connects every node to its k closest neighbors. While sparser graphs are generally preferred for computational and memory efficiency, reducing k or ϵ increases the risk of producing *disconnected* graphs (Belkin & Niyogi, 2003). This risk comes from the fact that both methods use only *local* proximity information and neither guarantees *global* connectivity. Theoretical analysis confirms this concern: a k -NN is asymptotically connected with high probability only when $k \geq 5.1774 \cdot \log N$ (Xue & Kumar, 2004), which already exceeds $k = 30$ for as few as $N = 300$ data points, well above the values typically used in practice.

Disconnected components are problematic for a range of downstream tasks. For example, in spectral clustering, each connected component can only be assigned to a single cluster; when the number of components equals or exceeds the number of desired clusters, the clustering becomes trivial, and no similarity-based criterion can improve it. In addition to clustering, disconnected graphs negatively affect other problems, such as item-based recommender systems, because unreachable items cannot be recommended (Seyerlehner et al., 2009).

In this work, we address this fundamental limitation by proposing a simple modification of the standard k -NN graph construction that guarantees a connected graph for any value of k , and apply it to spectral clustering of texts. The method proceeds incrementally: nodes are added one at a time, and each new node is connected to its k -nearest neighbors among the nodes already in the graph, ensuring that every insertion preserves connectivity. We introduce the graph-construction setting and illustrate the disconnection problem on the 20 Newsgroups dataset. We evaluate the resulting graphs on a downstream spectral clustering task using Laplacian eigenmaps (Belkin & Niyogi, 2003) and compare against the standard k -NN across two variants of six text-clustering datasets. Beyond the connectivity guarantee, the construction is efficient: it never forms the dense $N \times N$ distance matrix that exact minimum-spanning-tree (MST) repairs require, making it 23–44× faster to build than the exact MST-augmented alternatives at a fraction of the memory, and its advantage in the

sparse regime persists even when the standard graph is repaired with an MST (Sects. 6.2.2 and 6.2.4). Our approach is summarized in Fig. 1.

The paper is organized as follows. Section 2 reviews related work. Section 3 provides a motivating example and quantifies disconnected components on realistic text data. Section 4 presents the proposed incremental graph construction algorithm. Section 5 describes the experimental setup and Sect. 6 presents the experimental results. Section 7 concludes by presenting the directions for future work.

2 Background and Related Work

There are several ways to construct a graph representing the data in some metric space, where close nodes shall be connected, encoding the metric structure in graph connections. Two most popular ways are neighborhood graphs constructed from ϵ -distance and from k -nearest neighbors, as described in (Belkin & Niyogi, 2003). Constructing a graph from ϵ -neighborhood requires a choice of distance parameter ϵ : any two nodes with a distance less

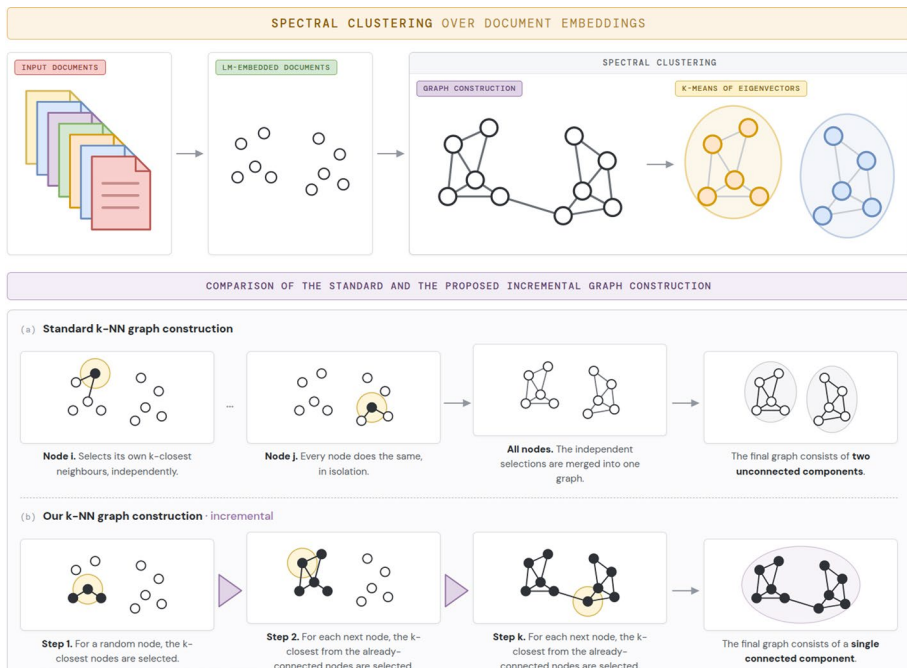


Fig. 1 Overview. Top: the spectral-clustering pipeline—input documents are embedded by a language model, a neighborhood graph is constructed over the embeddings (the step studied in this paper), and spectral clustering assigns labels by running k -means on the graph's leading eigenvectors. Bottom: the standard k -NN construction (a) versus the proposed incremental construction (b) on the same set of points; in every panel the yellow highlight marks the current node and its k -nearest-neighbor search. (a) In standard k -NN every node selects its k nearest neighbors independently over the whole set, and merging these independent selections can leave the graph with several disconnected components. (b) The incremental construction inserts nodes one at a time and links each new node to its k nearest *already-connected* nodes (filled); because the first node of a new region can only attach to earlier nodes, the graph remains a single connected component

than ϵ are connected (also called ϵ -radius). The downside is that connectedness is sensitive to ϵ : too small yields disconnected components, while too large yields many connections that can impact running time and downstream performance. For k -NN, every node is connected to its k nearest neighbors; this requires choosing k and tends to produce connected graphs, but loses the distance-based geometric interpretation.

2.1 Challenges with High-Dimensional Embeddings

Deep-learning derived embeddings often use several hundred to several thousand dimensions, which introduces challenges to their analysis. In order to compare objects represented with dense embeddings, the usual choice is cosine distance, while Euclidean distance is avoided: cosine distance is not a proper metric (triangle inequality does not hold), and Euclidean distance in high-dimensional spaces loses geometric interpretation and exhibits unintuitive phenomena (Aggarwal et al., 2001). Cosine distance scales distances to a range with maximum 2, making the ϵ parameter more sensitive to small changes. Theoretical results (Beyer et al., 1999) show poor discrimination between nearest and furthest points in high dimensions (Aggarwal et al., 2001), making ϵ -based selection unstable; k -NN was meant to address the difficulty of choosing ϵ (Belkin & Niyogi, 2003). High-dimensional neighborhoods are further distorted by *hubness*: a few points appear in a disproportionate number of nearest-neighbor lists while others become isolated “anti-hubs” (Radovanović et al., 2010), which skews degree distributions and aggravates the disconnection of k -NN graphs. However, both strategies may still generate multiple disconnected components, which is highly problematic for graph-based clustering.

2.2 Neighborhood Graphs and Applications to Clustering

Representing relationships as graph connections transforms complex geometry to graph topology (Tenenbaum et al., 2000) and enables tools from graph theory and complex network analysis (Liu & Barahona, 2020). Neighborhood graphs based on ϵ -distance and k -NN connect points by a distance threshold or to their k nearest neighbors. A more recent method is continuous k -NN (Ck-NN) (Berry & Sauer, 2019), but it can only be induced on a proper distance metric; in contrast, our work tries to induce a neighborhood graph even when a proper metric is not usable. Such similarity graphs and the embeddings learned over them support a wide range of downstream applications, from risk and behavior identification in social networks (Corizzo et al., 2023) to fault diagnosis of industrial processes (Dong et al., 2025); a broad survey of heterogeneous-graph embedding methods and their applications is given in Wang et al. (2023). In each case, a well-constructed and connected neighborhood graph is a prerequisite when the input data is not natively relational.

2.3 The Wider Clustering Landscape

Text clustering spans several algorithmic families beyond spectral methods (Aggarwal & Zhai, 2012). Centroid-based algorithms such as k -means (Macqueen, 1967), including the spherical variant tailored to cosine similarity (Dhillon & Modha, 2001), are efficient but presuppose compact, roughly isotropic clusters in the original high-dimensional space. Density-based methods such as DBSCAN (Ester et al., 1996) and HDBSCAN (Campello et al.,

2013; McInnes et al., 2017) make no shape assumptions but depend on density estimates that deteriorate in high-dimensional embedding spaces, an effect we also observe empirically (Sect. 6.1). Hierarchical agglomerative clustering (Murtagh & Contreras, 2012) provides multi-resolution structure at a quadratic cost, while deep clustering methods learn the representation and the cluster assignment jointly (Tsitsulin et al., 2023; Xie et al., 2016). Closest to our setting are approaches that cluster over an explicit neighborhood graph: modularity-based community detection such as Louvain (Blondel et al., 2008) and Leiden (Traag et al., 2019), community detection over embedding similarity (Škrlj et al., 2020), and widely used text-clustering pipelines such as BERTopic (Grootendorst, 2022) and Top2Vec (Angelov, 2020), which embed documents, reduce dimensionality with UMAP (itself built on a k -NN graph), and cluster the result with HDBSCAN. Spectral clustering (Luxburg, 2007; Ng et al., 2001) shares the neighborhood-graph substrate of this last family while accommodating non-convex clusters and offering a well-understood connection between the graph spectrum and cluster structure. Notably, every graph-based method in this landscape inherits the connectivity of the underlying neighborhood graph: community detection cannot merge disconnected components any more than spectral clustering can, and Leiden was motivated in part by repairing badly connected communities (Traag et al., 2019). A construction that is connected by design is therefore of interest across this whole family; we evaluate it in the spectral pipeline, where disconnection is most immediately harmful (Sect. 3).

Among global connectivity-based approaches, popular methods include MST-based constructions such as k NN-MST (Veenstra et al., 2016) and variants like PMST (Zemel & Carreira-Perpiñán, 2004) and RMST (Beguerisse-Díaz et al., 2013). PMST relies on Euclidean distances and random shifts, making it applicable only to low-dimensional data, while RMST performs comparisons and arithmetic on weights that are unintuitive under cosine distance. Although k NN-MST is a simple extension of k -NN by adding an MST, it efficiently resolves disconnected components and does not assume metric space or low-dimensional features. In contrast to the above, our method is applicable to undirected graphs, works in both low- and high-dimensional spaces, and is conceptually simple: it uses only k NN operations (no global features), enabling simpler implementation and faster processing.

2.4 Other Connected and Robust Neighborhood-Graph Variants

Beyond MST augmentation, several graph-construction schemes aim to improve connectivity or robustness, and it is useful to position our method against them. *Mutual* and *symmetric* k -NN graphs change which of the asymmetric k -NN edges are retained (Maier et al., 2009): the mutual variant keeps an edge only when each endpoint lies in the other's neighbor list (sparser, and more prone to disconnection at small k), while the symmetric variant keeps their union (denser); neither guarantees a single connected component. *Shared-nearest-neighbor* (SNN) graphs reweight edges by the overlap between neighbor lists (Jarvis & Patrick, 1973), improving robustness under varying local density but still inheriting the connectivity of the underlying k -NN graph. *Adaptive local-scaling* (self-tuning) kernels set a per-point bandwidth from local neighbor distances (Zelnik-Manor & Perona, 2004), which addresses heterogeneous scales rather than global connectivity. *Graph sparsification* methods, such as effective-resistance sparsifiers (Spielman & Srivastava, 2008), instead thin an already dense graph while preserving its spectral properties; they address the opposite regime to ours, which builds a sparse yet connected graph directly. Classical *proximity graphs*, notably the

relative neighborhood graph and the Gabriel graph (Toussaint, 1980), are connected by construction, but they are defined through geometric empty-region predicates that presuppose a proper metric and degrade in high dimensions. Modern manifold-learning pipelines such as UMAP (McInnes et al., 2018) build a k -NN graph and then impose a *local-connectivity* constraint so that every point links to at least its nearest neighbor, which mitigates but does not eliminate disconnection; moreover, that graph is constructed for UMAP's fuzzy-simplicial embedding objective rather than for spectral clustering. In contrast to all of these, our incremental construction guarantees connectivity by construction for every k , relies only on k -NN operations without global weights or a proper metric, and supports single-node incremental updates.

3 Demonstrating Disconnected Components on Realistic Data

We demonstrate the problems with neighborhood graphs constructed from textual embeddings on the well-known 20Newsgroups dataset¹ collection of approximately 20,000 documents grouped into 20 different topics. We use this data as a motivating example to construct neighborhood graphs, studying different parameters and the numbers of disconnected components. The examples in this section use 7,532 documents that are part of the test set. We apply the recommended preprocessing: removal of headers, signature blocks, and quotation blocks. The preprocessed documents are encoded using the SentenceTransformers (Reimers & Gurevych, 2019) model (all-MiniLM-L12-v2) that generates 384-dimensional embeddings. We use the cosine distance on these vectors to induce both ϵ -distance and k -NN neighborhood graphs.

3.1 Cosine Distance ϵ -Neighborhood Graph

In order to analyze the ϵ -neighborhood strategy, we find the minimal ϵ distance (ϵ_0) that ensures connectedness and test several properties of the resulting graph. In order to determine ϵ_0 , we calculate the cosine similarity between all pairs from the datasets and apply a bisection algorithm over the range of parameter ϵ . This determined a constant $\epsilon_0 = 0.7694$, specific for this dataset and the model used to encode the data. This distance corresponds to a Cosine similarity of $(1 - \epsilon_0) = 0.2306$. Table 1 presents some properties of this graph, the information on the ϵ distance used to infer the graph, the number of components in the resulting graph, the size of the largest component, the number of edges in this graph and

Table 1 Properties of ϵ neighborhood graphs for different values of ϵ

ϵ	Num comp	Max comp. size	Graph edges	Digraph edges
$\epsilon_0 - 10\%$	19	7511	404,956	809,912
$\epsilon_0 - 5\%$	3	7530	636,040	1,272,080
ϵ_0	1	7532	1,009,318	2,018,636
$\epsilon_0 + 5\%$	1	7532	1,622,168	3,244,336
$\epsilon_0 + 10\%$	1	7532	2,645,373	5,290,746

A minimal number of graph edges to satisfy the connectedness of the graph is achieved with $\epsilon = \epsilon_0$. A graph contains disconnected components if the number of components is greater than 1

¹ Available from <http://qwone.com/~jason/20Newsgroups/>

the number of edges in a symmetric (undirected) graph. In addition to properties of a graph induced with ϵ_0 distance, we provide information on graphs using ϵ that differ by 5% and 10% from ϵ_0 .

A graph generated from this dataset with a minimal distance required to retain graph connectivity requires around 1 million graph edges. For the calculation of Laplacian eigenmaps, we are interested in a graph with a sparse symmetric adjacency matrix. Such a matrix would contain 2 million entries (non-zero elements). The sparsity of the adjacency matrix is desired as it enables us to encode the relations within data using less information and speed up any subsequent processing step. From Table 1, it can easily be seen that using ϵ only 5% larger than the minimal required increases the number of edges by more than 60%.

3.2 Cosine-Distance Based k -Nearest Neighbor Graph

Using the same setting as for the ϵ -neighborhood graph, we generate k -NN neighborhood graphs by increasing the parameter k and observing the number of connected components. A summary of results is presented in Table 2. For each row, the table includes the value of parameter k , the number of components, the number of nodes in the largest connected component, the number of connected pairs, and the number of edges in a symmetric adjacency matrix. The fully connected directed graph requires $k = 5$ nearest neighbors and contains more than 57,000 edges. The k -nearest neighbors strategy was easier to tune to get the connected graph. The resulting graph is much sparser compared to the ϵ -neighborhood strategy, containing less than 3% of its number of edges. For many downstream tasks, this sparseness would translate into significantly faster processing.

4 Incremental k -NN Neighborhood Graph Construction

In this section, we describe the proposed modified k -NN neighborhood graph construction algorithm with two important features. First, it avoids disconnected components, a problem described in detail in Sect. 3. Second, it allows the addition of new nodes after the graph is built, so new documents can be inserted without reconstructing the existing graph.

The pseudocode of the proposed incremental k -NN neighborhood graph construction algorithm is outlined in Algorithm 1. The algorithm receives as the input the same parameters as a regular k -NN neighborhood graph construction algorithm, i.e. a set of N vectors and parameter k . It returns a generated graph, analogous to the k -NN neighborhood graph, described in Sect. 3.2. The algorithm sequentially processes each input vector. For each input, it considers only nodes already present in the graph for k -nearest neighbors. Nodes in the graph are denoted with the set of vertices V and connections between them E .

Table 2 Properties of k -NN neighborhood graphs for different values of parameter k

k	Num comp	Max comp. size	Graph edges	Digraph edges
1	1,505	216	7,532	12,054
2	49	7,342	15,064	23,470
3	6	7,507	22,596	34,734
4	2	7,527	30,128	45,978
5	1	7,532	37,660	57,132

A graph contains disconnected components if the number of components is greater than 1

4.1 Preliminaries and Notation

Let $X = \{x_1, \dots, x_N\}$ with $x_i \in \mathbb{R}^d$ denote the document embeddings, compared by cosine similarity $\text{sim}(x_i, x_j) = \langle x_i, x_j \rangle / (\|x_i\| \|x_j\|)$. We first make precise the neighborhood-selection and graph objects on which both the standard and the proposed constructions operate.

Definition 1 (*k-nearest-neighbor selector*) For a point $x \in X$, a candidate set $V' \subseteq X \setminus \{x\}$ with $|V'| \geq k$, and an integer $k \geq 1$, the selector $\text{kNN}(x, V', k)$ returns the subset of the k elements of V' with the largest cosine similarity to x , with ties broken by index.

Definition 2 (*Neighborhood graph and affinity*) A neighborhood graph over X is $G = (V, E)$ with $V = X$ and a (possibly asymmetric) binary adjacency $B \in \{0, 1\}^{N \times N}$, where $B_{ij} = 1$ iff $x_j \in \text{kNN}(x_i, V'_i, k)$ for a node-specific candidate set V'_i . Spectral clustering operates on a symmetric affinity matrix A , obtained either from the connection-based scheme $A = \frac{1}{2}(B + B^\top)$ or from a distance-based (Gaussian) kernel; both schemes are detailed at the end of this section. The edge set is $E = \{(i, j) : A_{ij} > 0\}$, and G is *connected* when the graph with adjacency A has a single connected component.

The two constructions differ only in the candidate set V'_i . The *standard* k -NN graph uses the full set $V'_i = X \setminus \{x_i\}$ for every node (a global neighbor search). The *incremental* construction of Algorithm 1 fixes an insertion order $x_1, \dots, x_N$ and restricts the candidate set of the t -th inserted node to the nodes already present, $V'_{x_t} = \{x_1, \dots, x_{t-1}\}$. This restriction alone, searching only among previously inserted nodes, guarantees a connected graph for any k (Theorem 1) while inducing only local changes to the adjacency matrix on insertion.

```

1: Inputs:
    $X = \{x_i; i \in 1, \dots, N\}$                                 ▷ N input vectors
    $k \in \{1 \dots N - 1\}$                                        ▷ the number of nearest neighbors
2: Initialize:
    $V \leftarrow \{x_1 \dots x_k\}$                                 ▷ Initial set of  $k$  vertices
    $E \leftarrow \emptyset$                                        ▷ Initial empty set of edges
3: for  $t = k+1$  to  $N$  do                                       ▷ Iteratively add the remaining vertices.
4:    $Q \leftarrow k\text{-NN}(x_t, V, k)$                                ▷ Find  $k$  nearest neighbors of  $x_t$  in  $V$ .
5:    $V \leftarrow V \cup \{x_t\}$                                    ▷ Add  $x_t$  to  $V$ .
6:   for  $q \in Q$  do                                           ▷ Add edges from  $x_t$  to all the nearest nodes in  $Q$ .
7:      $E \leftarrow E \cup \{(q, x_t)\}$ 
8:   end for
9: end for
10: return  $G(V, E)$                                            ▷ Graph with  $k(N - k)$  edges.

```

Algorithm 1 Incremental k Nearest Neighborhood Graph Construction

A difference between constructing a regular k -NN neighborhood graph and an incremental Algorithm 1 can be seen in Line 4 of the algorithm. While the standard k -NN neighborhood graph is constructed by searching for k nearest neighbors among all possible nodes, Algorithm 1 considers only nodes already added to the graph.

4.2 Properties of the Algorithm

When compared to the regular k -NN neighborhood graph construction based on an exact k -NN search, the construction of a graph with the proposed algorithm requires fewer comparisons. This is due to Algorithm 1 performing search only on a restricted set of nodes instead of the full set of nodes as with standard k -NN algorithm.

Algorithm 1 returns a directed selection graph, in which each edge is oriented from a selected neighbor q to the inserted node x_i . All connectivity statements in this paper refer to the undirected support of this graph, which coincides with the graph induced by the symmetric affinity matrix A of Definition 2; the theorem below is stated in these terms.

Theorem 1 *For every $k \geq 1$ and every insertion order of the input vectors, the graph returned by Algorithm 1 is connected.*

Proof The proof is by induction on the number of inserted vertices.

Base case. The initial set of k vertices carries no edges. The first inserted vertex x_{k+1} is linked to its k nearest neighbors among the vertices already present, i.e., to all k of them, which creates a single connected component containing all $k + 1$ vertices.

Inductive step. Assume the graph is connected after the insertion of x_{i-1} for some $i > k + 1$. The next vertex x_i is linked to $k \geq 1$ vertices of this connected component, which extends the component by x_i , so the graph remains connected.

Consequence. The final graph on all N vertices consists of a single connected component. $\square$

In the standard k -NN neighborhood graph, the consequence of adding a single additional node to the already constructed graph may be a large reconfiguration, as many existing nodes can be connected with the newly added node. This prevents efficient incremental expansion with new nodes and requires graph recreation.

In contrast, graphs constructed with Algorithm 1 can be expanded efficiently, because the newly added node induces only local changes in the graph adjacency matrix. Those changes are restricted to the row and column corresponding to the newly added node. This can be exploited in applications where all nodes are not available at the start, like in the processing of streaming data.

4.3 The Weights of the Adjacency Matrix

The Laplacian eigenmaps algorithm interprets entries in the graph adjacency matrix as the affinity between nodes. The affinity on the scale $[0, 1]$ describes a spectrum of node closeness where 0 and 1 represent very distant and very close nodes. In Belkin and Niyogi (2003), two schemes for the choice of affinities are proposed, one based on the presence of connections between nodes and the other taking into account the distances between nodes.

4.4 Connection-Based Node Affinity

A simple choice to construct the affinity matrix is to assign 1 to connections between nodes within k -NN neighborhood and 0 otherwise. The affinity matrix for the Laplacian eigen-

maps has to be symmetric, while k -NN neighborhood graphs may have asymmetrical adjacency matrices. To remedy this, a popular approach is to average the adjacency matrix with its own transpose.

4.5 Gaussian Kernel-Based Node Affinity

Elements of affinity matrix can also be computed based on the kernel function defined as $\alpha_{ij} = e^{-\frac{\|x_i - x_j\|^2}{4t}}$ with an additional hyperparameter $t \geq 0$. This variant of affinity matrix is inherently symmetric and is suitable for graphs based on the ϵ -distance while the connectivity-based affinity matrix is usually used with k -NN neighborhoods.

5 Experimental Setting

Manifold learning approaches, including Laplacian eigenmaps, lack a natural measure to assess the quality of learned embedding. Existing research provides an answer for the evaluation of learned manifold only in the setting without noise (Zhang et al., 2012). However, the Laplacian eigenmaps have a natural application to clustering that can be used to verify that embeddings are encoding relevant information. In our evaluation, we address the following aspects. First, we verify that the proposed approach does not introduce large performance regression when compared to the standard k -NN neighborhood graph. Second, we test if our approach improves the clustering results in a restricted setting, using small k , where disconnected components would otherwise occur. Third, we gain insight into the amount of information lost when compared to the original high-dimensional embeddings. Finally, due to the inherent dependence on the ordering of nodes in Algorithm 1 on a resulting graph, we are interested in the stability of the approach with respect to the ordering of the nodes.

We address the first two goals by running clustering, configured to use our approach and using a k -NN neighborhood graph. With a low value of parameter k , we expect more disconnected components and a larger difference to our approach. With higher values of k , both clustering evaluations will likely run on a connected graph. In this scenario, our approach does not benefit the task and may negatively influence the results due to fewer available nodes. This will enable us to quantify the negative aspects of our approach when used on a dataset that already produces connected neighborhood graphs. Although the amount of information lost by encoding high-dimensional embeddings to low-dimensional ones through Laplacian eigenmaps cannot be measured directly, we can evaluate the full-graph and our incremental approach on the same downstream task and quantify the difference in performance. In particular, we compare the clustering performance of low-dimensional embeddings produced by our approach with the clustering performance of original high-dimensional embeddings using the K-means algorithm.

To measure variance, due to the input node order, we run spectral clustering using a neighborhood graph generated by Algorithm 1 for each dataset ten times, each time randomizing the ordering of nodes, using connection-based node affinity. Due to the guaranteed connectedness of the graph, the first dimension of all embedding vectors is collapsed to a constant value, so we use an additional Laplacian eigenvector instead. The QR factorization

algorithm used for clustering of spectral embeddings was likewise amended to ignore the first (constant-value) dimension when assigning cluster labels to the embedding vectors.

In the following subsections, we provide a short overview of the evaluation datasets, methods used to infer document representations, neighborhood graph construction, and clustering metrics. The results of experiments are discussed in Sect. 6.

5.1 Datasets

We evaluate our approach on clustering datasets included in the Massive Text Embedding Benchmark (Muennighoff et al., 2023) (MTEB).² The datasets come from six sources, and are assembled in two variants; the first variant contains only titles, and the second variant of a dataset uses a concatenation of the title and body of a document. In MTEB, these two variants are called sentence-to-sentence (S2S) and paragraph-to-paragraph (P2P), respectively. Each dataset is partitioned into several sets, where each set contains a subset of labels such that both fine- and coarse-grained differences are evaluated. In total, there are 182 different clustering problems. Detailed statistics on the datasets can be found in Table 3. Additional details on the datasets are available in Muennighoff et al. (2023).

5.2 Document Representation

SentenceTransformers (Reimers & Gurevych, 2019) is a unified framework for sentence, text and image embeddings that uses Siamese and triplet network models to produce semantically meaningful high-dimensional data representations. For all our experiments, we use text embeddings generated from `all-MiniLM-L12-v2` model. It encodes input text limited to 256 tokens to an embedding of 384 dimensions. Laplacian eigenmaps algorithm was used to transform those embeddings to a low-dimensional representation with dimensionality matching the number of clusters.

5.3 Metrics

As per MTEB (Muennighoff et al., 2023) benchmarking suite, the clustering performance is evaluated using *V-measure* (Rosenberg & Hirschberg, 2007) (V_1) score. *Homogeneity* h is

Table 3 An overview of text datasets used in the evaluation

Dataset	S2S						P2P					
	Title	Body	docs	avg. len	P	C	docs	avg. len	P	C		
ArXiv	✓	✓	732723	74.00	31	171	732723	1009.90	31	171		
BioRxiv	✓	✓	75000	101.60	10	26	75000	1666.20	10	26		
MedRxiv	✓	✓	37500	114.70	10	51	37500	1981.20	10	51		
Reddit	✓	✓	420464	64.70	25	50	459399	727.70	10	87		
StackExchange	✓	✓	373850	57.00	25	50	75000	1090.70	10	385		
TwentyNewsgroups	✓	—	59545	32.00	10	20	—	—	—	—		

We report the number of documents, average document length, number of partitions in column P, and maximum number of clusters within the problems in column C. The statistics are presented for both sentence-to-sentence (S2S) and paragraph-to-paragraph (P2P) tasks

²Although MTEB comes with its own evaluation based on MiniBatchKMeans clustering, we do not use it for our purposes and evaluate our approach using spectral clustering with QR decomposition.

used to evaluate how close a given clustering is to an ideal clustering where each cluster only contains items of a single class. *Completeness* c evaluates how close a given clustering is to perfectly complete clustering where all members of a single class are assigned to a single cluster (achieving completeness $c = 1$). Putting all items into a single cluster would achieve $c = 1$, but the *homogeneity* would be zero ($h = 0$). A different type of degenerate clustering, assigning each item to a separate cluster, will have a perfect *homogeneity* ($h = 1$), but the *completeness* will be zero. The harmonic mean of those two opposite aspects is called *V-measure* (V_1), also known as Normalized Mutual Information (NMI). Similarly to the familiar F-score, it has a parameter β where larger values give more weight to completeness.

$$V_{\beta} = (1 + \beta) \frac{h \cdot c}{\beta \cdot h + c}$$

For the baseline k -NN neighborhood graph, the evaluation also checks for any disconnected components in the neighborhood graph. This check is not needed for our incremental solution as the graph is guaranteed to be connected.

6 Results

This section presents the results of spectral clustering and K-means clustering. For graph creation, we use our incremental approach or the standard k -NN neighborhood approach. Following Muennighoff et al. (2023), the results of different partitions from the same data source are averaged.

6.1 Main Results

Figure 2 and Table 4 show V-measure results comparing our incremental approach (*Ours LD*) and standard k -NN neighborhood (*k-NN LD*). We use high-dimensional embeddings generated from the SentenceTransformer model described in Sect. 5.2 and apply K-means clustering in order to have a reference point for an upper bound of the clustering performance on the low-dimensional spectral embeddings. This value is shown in Table 4 under the row *K-means HD*. Standard clustering shows low performance with low values of k and, in general, recovers most of the final score above $k = 8$ when a further increase in k brings only diminishing returns. Spectral clustering using our neighborhood graph converges much faster to the limit of its performance. Already at $k = 3$ the performance is close to the top-score achieved for a dataset. Overall, our incremental approach achieved consistently better scores across all measured values of k on four out of eleven dataset variants, while in others, it outperformed the standard approach for small values of k .

When comparing the two approaches to clustering, the largest differences in performance are seen on the *TwentyNewsgroups*³ dataset where our approach achieves higher score. This is consistent with the large number of disconnected components in the standard k -NN neighborhood graph. On every dataset evaluated, there was at least one split of the data with disconnected components using $k = 5$. Some dataset partitions within *TwentyNewsgroups*

³TwentyNewsgroups is a variant of 20Newsgroups datasets from MTEB where only subject headers are available.

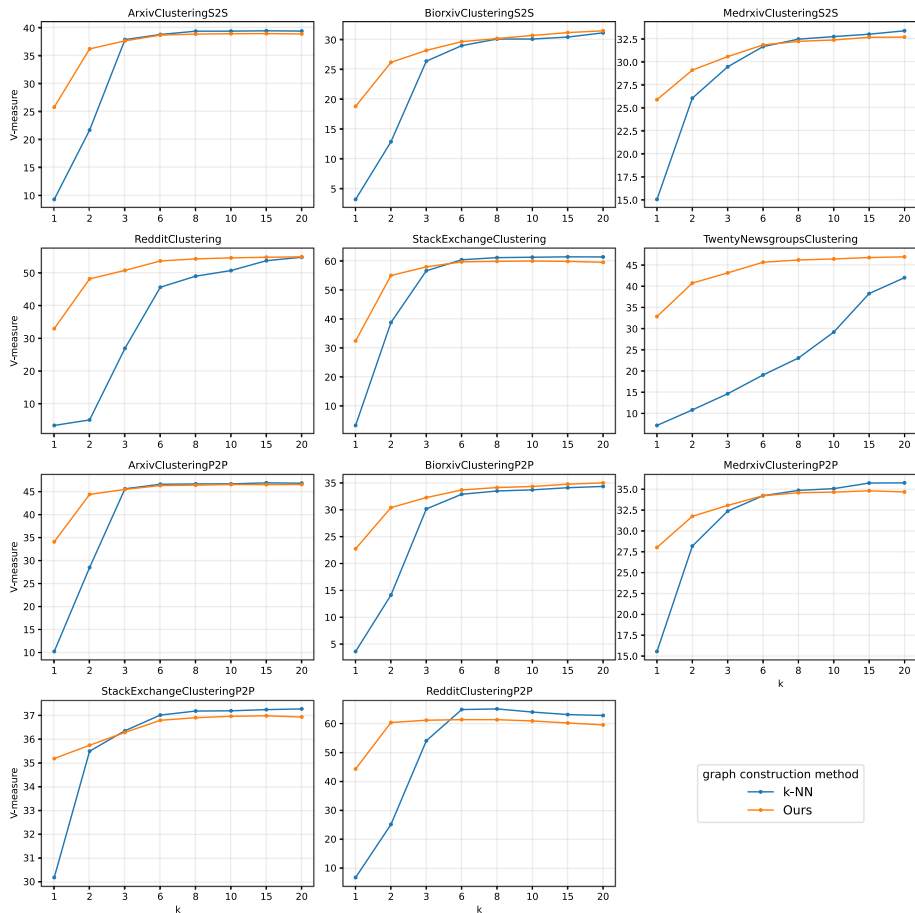


Fig. 2 Clustering performance of embeddings induced using incremental k -NN (Ours) and standard k -NN for a range of parameter k ; the values are those of Table 4. In Ours the graph is connected for every k by construction, while the standard k -NN graph may contain disconnected components at small k (Sect. 3). Results for sentence-to-sentence (S2S) tasks are shown in the two top rows and for paragraph-to-paragraph (P2P) tasks below them

dataset exhibited disconnected components using $k = 15$, and *Reddit* (sentence-to-sentence) even with $k = 20$.

Table 4 also provides a comparison with K-means clustering. Note an important difference between spectral and K-means clustering, namely the spectral clustering uses Laplacian eigenmaps-based dimensionality reduction, and the clustering results show performance on the low-dimensional space. Results for K-means are obtained by clustering directly in the high-dimensional space. By reducing the dimensionality of the data, one loses some information present in the high-dimensional space, and somewhat inhibited clustering performance is expected. As can be seen from Table 4, this is not always the case, and for some datasets, our approach exhibits improvement in the clustering metrics. This can be explained by the ability of the spectral clustering method to perform well even with irregularly shaped clusters, while K-means has much stricter assumptions on the cluster shape.

Table 4 V_1 scores (in percentages, larger is better) for spectral clustering of low-dimensional embeddings (LD) using standard k -NN neighborhood graph and our incremental approach

		S2S			P2P							
	k	Arx	Bio	Med	Red	SE	20NG	Arx	Bio	Med	Red	SE
k-NN LD	1	9.33	3.24	15.07	3.43	3.31	7.19	10.23	3.63	15.55	6.67	30.17
	2	21.73	12.91	26.09	5.11	38.83	10.85	28.49	14.13	28.18	25.06	35.49
	3	37.93↑	26.43	29.51	26.99	56.67	14.67	45.61↑	30.16	32.37	54.03	36.35↑
	6	38.85↑	29.03	31.71	45.67	60.48↑	19.10	46.60↑	32.89	34.22–	64.86↑	37.01↑
	8	39.44↑	30.14	32.52↑	49.05	61.22↑	23.10	46.67↑	33.50	34.86↑	65.08↑	37.18↑
	10	39.45↑	30.14	32.80↑	50.77	61.36↑	29.22	46.69↑	33.71	35.07↑	63.99↑	37.19↑
Ours LD	15	39.51↑	30.47	33.06↑	53.79	61.49↑	38.32	46.90↑	34.11	35.74↑	63.12↑	37.24↑
	20	39.46↑	31.18	33.43↑	54.83	61.45↑	42.07	46.83↑	34.35	35.76↑	62.80↑	37.27↑
	1	25.85↑	18.85↑	25.93↑	33.02↑	32.48↑	32.89↑	34.09↑	22.74↑	28.02↑	44.28↑	35.18↑
	2	36.27↑	26.23↑	29.15↑	48.22↑	55.04↑	40.79↑	44.39↑	30.41↑	31.75↑	60.37↑	35.74↑
	3	37.71	28.23↑	30.62↑	50.82↑	58.03↑	43.20↑	45.48	32.27↑	33.06↑	61.16↑	36.28
	6	38.74	29.70↑	31.89↑	53.70↑	59.75	45.72↑	46.32	33.69↑	34.22–	61.36	36.79
K-means HD	8	38.92	30.22↑	32.28	54.34↑	59.94	46.25↑	46.40	34.14↑	34.58	61.35	36.90
	10	38.99	30.74↑	32.43	54.64↑	60.05	46.49↑	46.55	34.34↑	34.65	60.91	36.96
	15	39.01	31.22↑	32.72	54.83↑	59.93	46.82↑	46.52	34.77↑	34.81	60.20	36.98
	20	38.95	31.52↑	32.75	54.96↑	59.60	46.99↑	46.55	35.01↑	34.67	59.55	36.93
		38.24	33.99	32.73	52.87	54.85	48.67	46.76	38.39	34.63	55.53	37.08

An arrow is shown to mark the relative difference between two variants for the same value of k . Additionally, we provide results for the clustering of high-dimensional embeddings (HD) using the K-means algorithm

The proposed incremental algorithm uses a given ordering of the graph nodes, and the resulting neighborhood graph is dependent on this ordering. In Table 5, we analyze the stability of the clustering performance with regard to the ordering of processed graph nodes. The table shows an average standard deviation for a dataset calculated from ten runs of the clustering algorithm. The results show that even with very low values of k , like $k = 3$, the standard deviation of clustering performance is rarely above 1% and is often below 0.5%. We also show that increasing the value of k reduces the standard deviation of results for the clustering task. We also considered experiments with HDBSCAN (McInnes et al., 2017) clustering, but they are not included due to the very low quality of results. The low HDBSCAN scores may partly result from using Euclidean distance on normalized embeddings, since the reference implementation does not support cosine distance; we did not investigate this explanation further.

6.2 Ablation Study

In this paper, we have so far only used the latent embedding space of the `all-MiniLM-L12-v2` model. Although this model is robust, it has some downsides, such as the length of the context, the dimensionality of the embedding, and the number of parameters, making it an unsuitable candidate for larger and more complex documents. In Sect. 2, we have however introduced several works that improve clustering results by adding information from the MST graph. We are therefore interested if adding this information further improves the proposed approach. Beyond these two questions, we analyze which structural properties of the constructed graphs predict clustering quality, quantify the computational cost of all constructions, and study the sensitivity of the construction to the node-insertion order.

6.2.1 The Text Embedding Model

To evaluate the effects of different embedding models and the dimensionality of the embedding space, we use several model variants, described in Table 6. Our aim is to verify if the approach described in this paper is sensitive to the quality of high-dimensional embeddings or if Laplacian eigenmaps transformation to low-dimensional vectors already saturates the expressive power of this low-dimensional embedding space.

For each embedding model, we apply the proposed algorithm with the same setting as in the previous experiments and test it on all of the available datasets; we make the results available in Fig. 3. Across datasets, we observed enhanced performance with increasing k -values, with the notable exception of the *RedditClusteringP2P* dataset. This exception indicates that the benefit of denser graphs and of larger embedding models is dataset-dependent.

We next analyzed if there is a statistically significant difference between embedding models based on the Friedman rank test with Nemenyi Demšar (2006) post-hoc correction (refer to Fig. 4) at the standard $\alpha = 0.05$ significance level. We rank the models in increasing order of V-measure, therefore, higher rank means better performance. The test outcomes, yielding a p-value below 0.01 and a CD of 0.65, highlight significant performance disparities among larger models. Specifically, larger models (*bge-base-en-v1.5*, *gte-large*, *all-mpnet-base-v2*) achieve higher V-measure scores compared to the smaller *all-MiniLM-L12-v2* and *all-MiniLM-L6-v2* models.

Table 5 Standard deviation of clustering on neighborhood graphs created by our approach

	k	S2S										P2P					SE
		Arx	Bio	Med	Red	SE	20NG	Arx	Bio	Med	Red	Arx	Bio	Med	Red	SE	
Ours LD	1	0.84%	0.83%	0.52%	1.25%	1.42%	1.42%	1.06%	0.82%	0.55%	1.38%	0.82%	0.82%	0.55%	1.38%	0.21%	
	2	0.57%	0.77%	0.52%	1.03%	0.95%	1.33%	0.55%	0.66%	0.48%	0.86%	0.55%	0.66%	0.48%	0.86%	0.17%	
	3	0.45%	0.71%	0.45%	0.92%	0.81%	1.53%	0.47%	0.53%	0.40%	0.75%	0.47%	0.53%	0.40%	0.75%	0.15%	
	6	0.43%	0.59%	0.38%	0.63%	0.88%	0.86%	0.36%	0.55%	0.41%	0.60%	0.36%	0.55%	0.41%	0.60%	0.14%	
	8	0.34%	0.43%	0.41%	0.48%	0.86%	0.99%	0.34%	0.48%	0.38%	0.53%	0.34%	0.48%	0.38%	0.53%	0.14%	
	10	0.32%	0.46%	0.39%	0.55%	0.74%	0.75%	0.28%	0.42%	0.42%	0.54%	0.28%	0.42%	0.42%	0.54%	0.15%	
	15	0.26%	0.49%	0.35%	0.47%	0.70%	0.80%	0.28%	0.46%	0.40%	0.50%	0.28%	0.46%	0.40%	0.50%	0.15%	
	20	0.25%	0.39%	0.42%	0.36%	0.64%	0.61%	0.26%	0.37%	0.38%	0.60%	0.26%	0.37%	0.38%	0.60%	0.14%	

Table 6 Additional models used to evaluate the properties of the text embedding space

Model	Emb. Dimensions	Max tokens	Model size (GB)	Average V-measure
all-MiniLM-L6-v2 (Reimers & Gurevych, 2019)	384	256	0.09	0.41
all-Mini-LM-L12-v2 (Reimers & Gurevych, 2019)	384	256	0.13	0.40
all-mpnet-base-v2 (Reimers & Gurevych, 2019)	768	384	0.42	0.42
bge-base-en-v1.5 (Xiao et al., 2023)	768	512	0.44	0.44
gte-large (Li et al., 2023)	1024	512	0.67	0.44

6.2.2 Comparison with MST-Augmented Graphs

In our incremental approach, we process nodes sequentially, building a connected graph using only local nearest neighbor information. In Sect. 6.1, we compare our approach to a standard nearest neighborhood graph that also uses local nearest neighbor information in a greedy approach to graph construction with the downside of the graph being potentially unconnected. A minimum spanning tree (MST) and its variants are posed as a main alternative in the related work (Sect. 2) and were shown to improve the clustering.

To isolate the contribution of this global structure, we evaluate all four combinations of graph construction (standard k -NN and our incremental graph, denoted *Ours*) and MST augmentation, in which the edges of the MST are added to the affinity matrix of the base graph. This compares the two constructions both bare and under matched augmentation, rather than against an un-augmented reference only. Table 7 reports the resulting V_1 scores across all datasets and values of k .

Comparison under matched augmentation When both constructions receive the same MST augmentation, the incremental graph is stronger in the sparse regime, where connectivity is scarce. For $k \in \{1, 2, 3\}$, Ours+MST improves on k -NN+MST by 2.5 V_1 points on average, ahead on 9 of the 11 dataset variants (paired Wilcoxon signed-rank test over dataset-level means, to account for the dependence among results at different k ; $p = 0.019$). The gap is largest at $k = 1$ (+3.8 points, ahead on 10 of 11, $p = 0.007$). The two converge as k grows, and for $k \in \{15, 20\}$ k -NN+MST is marginally ahead (−0.7 points, ahead on 10 of 11, $p = 0.05$) as both approach the K-means upper bound of Table 4. All MST-augmented configurations and the Ours block of Table 7 are obtained within a common experimental framework; small differences from Table 4 (about one V_1 point) arise from independent node orderings and the exclusion of the five largest Reddit-P2P partitions. The incremental construction thus retains its low- k advantage even after the baseline receives the MST repair it requires for connectivity.

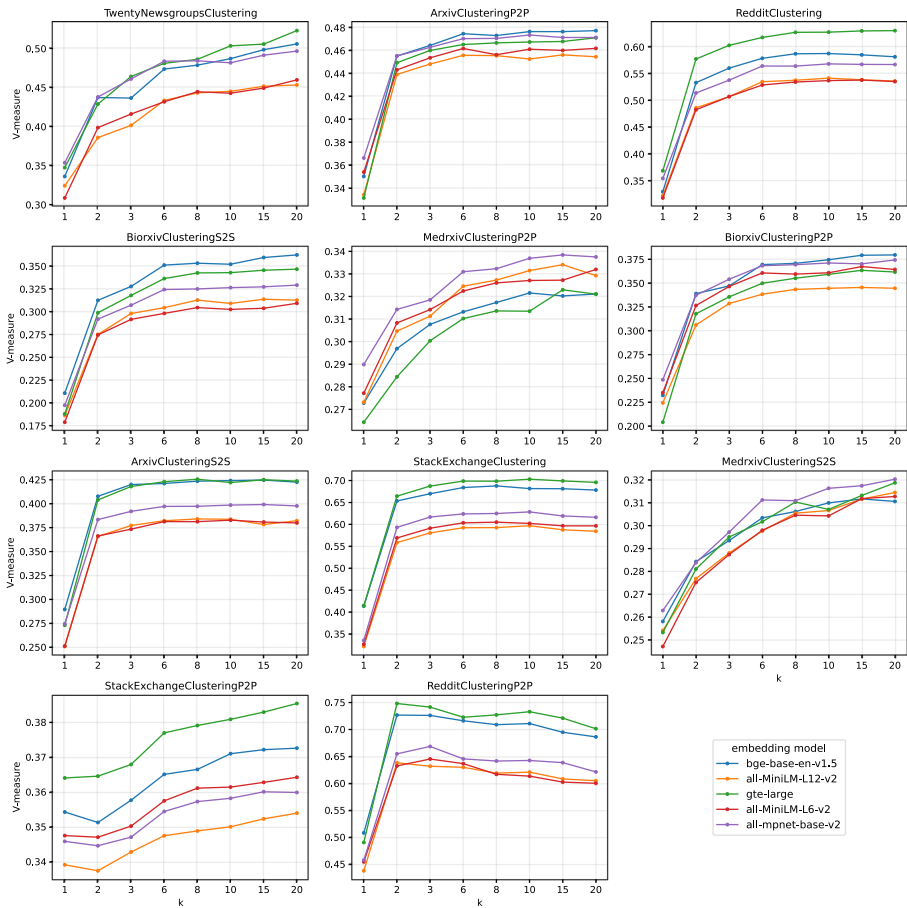


Fig. 3 The impact of different embedding models applied to different datasets

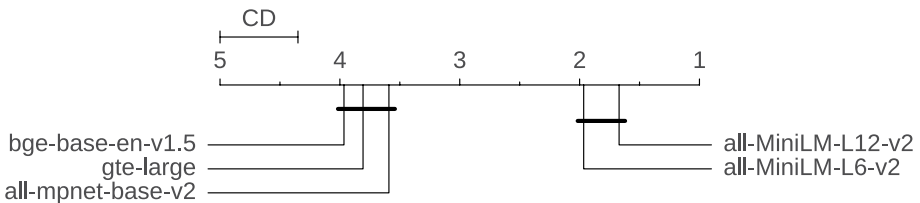


Fig. 4 The Nemenyi-Friedman test with critical distance of multiple embedding models across all datasets and values of k

Contribution of the MST to the incremental graph Comparing Ours+MST with the bare incremental graph isolates the effect of the global MST edges on our method. The MST yields a substantial improvement only at $k = 1$ (+8.6 V_1 points, ahead on 10 of 11 dataset variants; dataset-level Wilcoxon $p = 0.002$), where the graph is sparsest; at $k = 2$ the gain drops to +1.0 point, and for $k \geq 3$ it lies within the run-to-run variability of Table 5

Table 7 V_1 scores (in percentages, larger is better) for the standard k -NN graph and our incremental graph (Ours), each evaluated with and without minimum-spanning-tree (MST) augmentation, using *all-MiniLM-L12-v2* embeddings

	k	S2S					P2P					
		Arx	Bio	Med	Red	SE	20NG	Arx	Bio	Med	Red	SE
k-NN	1	9.3	3.2	15.1	3.4	3.3	7.2	10.2	3.6	15.6	6.7	30.2
	2	21.7	12.9	26.1	5.1	38.8	10.8	28.5	14.1	28.2	25.1	35.5
	3	37.9	26.4	29.5	27.0	56.7	14.7	45.6	30.2	32.4	54.0	36.4
	6	38.9	29.0	31.7	45.7	60.5	19.1	46.6	32.9	34.2	64.9	37.0
	8	39.4	30.1	32.5	49.0	61.2	23.1	46.7	33.5	34.9	65.1	37.2
	10	39.5	30.1	32.8	50.8	61.4	29.2	46.7	33.7	35.1	64.0	37.2
k-NN+MST	15	39.5	30.5	33.1	53.8	61.5	38.3	46.9	34.1	35.7	63.1	37.2
	20	39.5	31.2	33.4	54.8	61.5	42.1	46.8	34.4	35.8	62.8	37.3
	1	29.0	20.8	27.3	42.6	43.1	37.7	37.3	24.4	29.8	50.5	35.1
	2	37.0	25.9	27.7	41.3	54.4	32.9	44.6	30.0	30.2	64.7	34.6
	3	38.9	28.4	28.9	41.4	58.9	24.7	46.1	32.3	31.5	64.7	35.2
	6	39.3	30.8	30.6	47.9	61.2	19.0	46.5	34.0	33.3	64.0	36.0
Ours	8	39.4	31.1	31.5	50.5	61.5	21.7	46.5	34.5	34.0	63.3	36.2
	10	39.4	31.5	31.8	52.2	61.5	26.6	46.6	34.6	34.3	62.8	36.4
	15	39.2	31.9	32.4	54.5	61.5	33.5	46.7	34.9	34.8	61.8	36.6
	20	39.2	32.2	32.9	55.5	61.2	39.3	46.6	35.1	35.2	61.7	36.8
	1	25.1	18.4	25.4	32.6	31.7	32.3	33.1	22.1	27.3	43.2	34.0
	2	36.6	27.7	27.8	48.4	55.5	39.3	44.0	31.1	30.2	58.1	33.7
	3	37.6	29.3	28.7	51.0	57.9	41.3	45.0	32.6	31.4	58.7	34.1
	6	38.2	30.7	30.0	53.5	59.2	43.3	45.4	33.6	32.6	58.4	34.7
	8	38.2	30.9	30.5	53.9	59.2	44.0	45.5	33.9	32.8	57.9	34.9
	10	38.2	31.2	30.6	54.0	59.1	44.3	45.5	34.2	33.1	57.3	35.0
15	38.1	31.5	31.1	54.2	58.8	45.4	45.5	34.5	33.2	56.1	35.2	
20	38.0	31.5	31.4	53.9	58.2	45.2	45.5	34.7	33.3	55.6	35.3	

Table 7 (continued)

	k	S2S					P2P					
		Arx	Bio	Med	Red	SE	20NG	Arx	Bio	Med	Red	SE
Ours+MST	1	34.9	25.0	27.6	47.2	52.1	38.6	43.0	28.8	30.1	58.3	33.8
	2	37.2	28.5	28.9	50.5	57.0	40.1	44.7	31.9	31.4	59.3	34.1
	3	37.9	29.7	29.4	51.9	58.5	41.1	45.1	32.8	31.9	59.6	34.4
	6	38.1	30.8	30.1	53.7	59.3	43.3	45.4	33.7	32.7	58.2	34.8
	8	38.3	31.0	30.5	54.1	59.2	44.2	45.5	34.0	32.9	57.8	35.0
	10	38.2	31.1	30.8	54.2	59.3	44.5	45.6	34.4	33.2	57.4	35.1
	15	38.2	31.4	31.2	54.1	58.8	45.3	45.5	34.5	33.3	56.4	35.2
	20	37.9	31.6	31.4	53.9	58.4	45.8	45.5	34.7	33.3	55.5	35.3

Values are means over ten node orderings; S2S and P2P denote sentence- and paragraph-level tasks. The k -NN block restates the values of Table 4; the remaining blocks are computed within a common experimental framework, and small differences from Table 4 arise from independent node orderings and the Red-P2P exclusion (see text). In the k -NN+MST, Ours, and Ours+MST configurations, the Red-P2P mean excludes the five largest partitions ($> 46\,340$ nodes), which exceed the 32-bit index range of the sparse eigensolver

(mean $|\Delta| < 0.1$ point). The incremental construction already supplies the connectivity that a k -NN graph must recover through the MST, so for $k \geq 3$ the MST step can be omitted at no measurable cost in quality. Because our method never forms the dense $N \times N$ distance matrix the MST requires, omitting it also yields substantial savings in time and memory (Sect. 6.2.4).

To assess the practical significance of the MST contribution, we also compare Ours and Ours+MST with the Bayesian signed-rank test Benavoli et al. (2017), using a region of practical equivalence (ROPE) of 0.01 on the V-measure scale. The test is run separately at $k = 1$ and $k = 6$, with one paired sample per partition (the V-measure mean over the ten node orderings), so that the samples within each test are independent. Table 13 in Appendix C reports the posterior probabilities, and the two slices give a sharp picture that matches the per- k analysis above: at $k = 1$ the posterior places all probability mass on Ours+MST for ten of the eleven dataset variants (the exception is StackExchange-P2P, where equivalence dominates), while at $k = 6$ practical equivalence attains probability 1.000 on every dataset variant. Table 14 extends this to all evaluated k : equivalence is reached on most variants by $k = 3$ and on all of them from $k = 6$ on. The MST is thus practically beneficial only in the sparsest regime and practically irrelevant from $k = 6$ on, confirming that the construction does not depend on a global connectivity repair at the sparsity levels used in practice.

6.2.3 Comparing Graph Properties

Next, we examine which structural properties of the constructed graphs are associated with clustering quality. The analysis covers graphs produced by the proposed incremental algorithm (**base**) and their MST-enriched variants (**base+mst**).

We consider several graph- and node-level properties. Given a graph $G = (V, E)$, where V is the set of documents and E is the set of edges connecting vertices, we examine the following measures: graph density, assortativity, transitivity, local clustering coefficient, PageRank, and homophily. Definitions of these metrics are provided in Appendix A. We also include document-level statistics: average number of words (avg. $|\text{words}|$), average number of sentences (avg. $|\text{sentences}|$), and average number of characters (avg. doc. len).

Computing exact graph statistics for large graphs is computationally intensive. Following related work on Monte Carlo estimation over graphs (Alexopoulos & Fishman, 1991; Avrachenkov et al., 2007), we therefore use a Monte Carlo simulation of graphs, selecting the number of nodes/edges examined such that statistical accuracy falls within a 95% confidence interval to the second decimal place. For this analysis, we use `all-MiniLM-L12-v2` as the embedding model and compare both the original graph structure (*base*) and the MST-enriched graph structure (*base+mst*) (see Sect. 6.2.2). We compute the correlation between these properties and the V-measure.

In Fig. 5, for the *document-level statistics*, we find negative correlations of longer documents to V-measure, i.e. the number of characters (−0.21), words (−0.17), and sentences (−0.13). Node count (0.38) and edge count (0.58) correlate positively with the V-measure. Because the edge count is directly determined by both the dataset size and the neighborhood size k , these correlations do not isolate an independent effect of graph size and should be read descriptively rather than causally. Using the graph-level statistics, we see that graphs that exhibit higher assortativity (coefficient 0.51), transitivity (0.44) and local clustering (0.39) correlate positively with the V-measure. This is also true for homophily (0.66), which

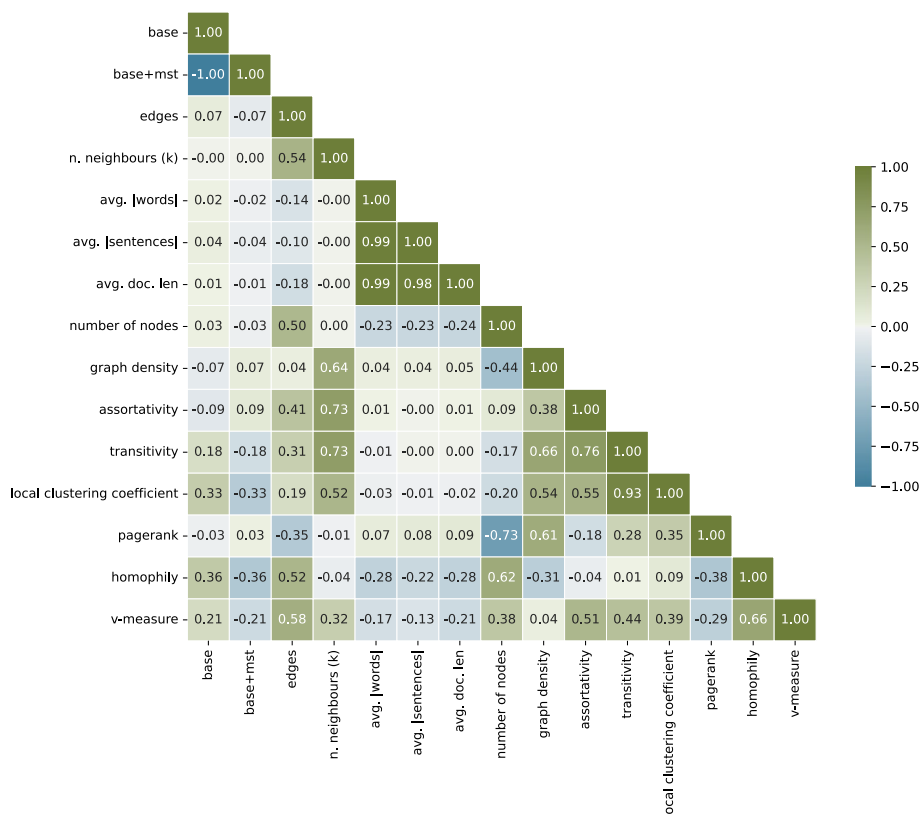


Fig. 5 Correlation of the graph-level and document-level properties and the *V-measure*

encompasses the above properties and the underlying intrinsic structure of the graph. We find that the higher the Pagerank, the lower the V-measure (correlation -0.29). One possible explanation is that high-PageRank documents act as cross-topic hubs whose many connections blur cluster boundaries; we do not test this hypothesis directly.

6.2.4 Computational Cost

A practical advantage of the incremental construction is that it never forms the dense $N \times N$ distance matrix that a minimum spanning tree requires. To quantify this, we constructed each graph from the embeddings and measured, separately, the total construction time, the spectral-solve time, the peak resident memory, and the number of non-zero affinity entries (nnz). Table 8 reports these quantities at $k = 10$ for all four constructions on the two largest single partitions we evaluate, a StackExchange S2S partition ($N \approx 11,000$) and an Arxiv P2P partition ($N \approx 25,000$), each run in isolation on a CPU node of the Vega EuroHPC system (two 64-core AMD EPYC 7H12 processors, 256 GB RAM), executed as a single process restricted to four BLAS/OpenMP threads, with the same embeddings and eigensolver; the exact library versions are pinned in the container image released with the code. Construction

Table 8 Computational cost at $k = 10$ for all four constructions on two representative partitions, each run in isolation on one CPU node

Dataset	Method	Construct (s)	Spectral (s)	Peak RSS (GB)	nnz
StackExchange (S2S), $N \approx 11k$	Ours	1.2	1.1	1.2	220,940
	Ours+MST	27.7	1.0	4.5	221,862
	k-NN	1.1	1.9	1.2	160,778
	k-NN+MST	28.2	1.9	4.5	160,778
Arxiv (P2P), $N \approx 25k$	Ours	3.5	1.7	3.3	499,800
	Ours+MST	155.9	1.6	20.2	502,240
	k-NN	4.5	2.3	3.2	370,336
	k-NN+MST	153.2	2.5	20.2	370,336

Construction time is the total time to build the affinity matrix from the embeddings, and includes the minimum-spanning-tree computation for the MST-augmented methods; we also report the spectral-solve time, the peak resident memory, and the number of non-zero affinity entries (nnz). All four graphs are connected at $k = 10$

time covers every step needed to turn the embeddings into the affinity matrix, so that the cost of building the MST is charged to the methods that use it.

Two effects dominate. First, the exact MST construction used by the augmented baselines is expensive to build: it requires the full pairwise-distance matrix and a spanning-tree pass over it, an $O(N^2)$ step that raises construction time from 1.2–3.5 s for the bare incremental graph to 28–156 s for either MST-augmented method—between 23× and 44× slower on these partitions—and raises peak memory from 1.2–3.3 GB to 4.5–20.2 GB. The incremental construction avoids this step entirely: it attains connectivity through the insertion order rather than through added global edges. Approximate nearest-neighbor or approximate-MST constructions could reduce the baseline’s cost at the price of exactness; our comparison covers the exact constructions used throughout the paper, timed as single wall-clock runs. Second, the spectral solve on the incremental graph is faster than on k -NN+MST (1.7 versus 2.5 s on Arxiv, 1.1 versus 1.9 s on StackExchange), which we attribute to the better-conditioned Laplacian of a graph that is connected by construction; adding the MST to our own graph does not slow its solve. Together with the quality results of Sect. 6.2.2, where the MST is redundant for $k \geq 3$, this makes the bare incremental graph the preferred configuration in practice: it matches the MST-augmented baselines in clustering quality at a small fraction of their construction time and memory.

6.2.5 Sensitivity to Node Ordering

Because the incremental graph depends on the order in which nodes are inserted (Sect. 6.1), we next examine whether structured, non-random orderings affect either clustering quality or the structure of the resulting graph. Table 5 already shows that the variance under random orderings is small; here we additionally evaluate the *seed* of the construction under two worst-case orderings, in which the first inserted nodes are deliberately concentrated before insertion continues in random order. In the *centroid* ordering the seed consists of the nodes closest to the global embedding centroid, forcing the initial receptive field into a tight ball at the center of the point cloud, the opposite of a well-spread seed. In the *class* ordering the seed is drawn entirely from a single ground-truth class, so the construction begins with documents of one class before any other document appears.

Table 9 shows that the ordering has a negligible effect on clustering quality: across the datasets shown and $k \in \{1, 2, 3, 6\}$, the centroid and class orderings differ from the random ordering by at most $0.4 V_1$ points, with a mean absolute difference of 0.08 point, below the seed-to-seed standard deviation reported in Table 5. Even the class ordering, the most extreme concentration of the seed a data stream could exhibit, does not measurably degrade clustering quality: a paired two-one-sided equivalence test (TOST), with one paired observation per dataset partition (the V-measure mean over the ten orderings and $k \in \{1, 2, 3, 6\}$; $n = 75$), confirms that both worst-case orderings are equivalent to the random ordering within a margin of $\pm 0.5 V_1$ points, a margin chosen to match the seed-to-seed variability reported in Table 5 ($p \leq 5 \times 10^{-21}$ for both); a paired difference test on the same observations detects no significant difference (Wilcoxon $p \geq 0.07$). These experiments provide evidence of robustness to strongly concentrated initial seeds; they do not, however, replace a direct evaluation on temporally ordered or gradually drifting document streams, in which an entire early segment of the input may come from a different distribution. The MTEB snapshots carry no per-document timestamps, so we leave such an evaluation to future work; the released code implements a `temporal` insertion strategy for corpora that do carry timestamps.

Clustering quality could in principle remain stable while the underlying graphs differ, for example if some orderings placed the initial nodes in regions of the embedding space where an effective graph is easier to construct. Table 10 therefore complements the quality-based analysis with descriptive statistics of the constructed graphs across the same ten random orderings. The edge count is invariant by construction: each inserted node contributes

Table 9 Effect of the node-insertion ordering on clustering quality (V_1 , percentages, left block) and on the structure of the constructed graphs (label homophily, right block)

Dataset	Ordering	V_1				label homophily			
		$k=1$	$k=2$	$k=3$	$k=6$	$k=1$	$k=2$	$k=3$	$k=6$
Bio-P2P	random	22.1	31.1	32.6	33.6	0.485	0.467	0.456	0.437
	centroid	22.1	31.1	32.5	33.7	0.485	0.467	0.455	0.436
	class	22.0	31.1	32.6	33.7	0.484	0.466	0.455	0.436
Bio-S2S	random	18.4	27.7	29.3	30.7	0.440	0.421	0.410	0.392
	centroid	18.4	27.7	29.3	30.6	0.440	0.420	0.410	0.392
	class	18.5	27.6	29.2	30.7	0.439	0.421	0.410	0.392
Med-P2P	random	27.3	30.2	31.4	32.6	0.428	0.404	0.390	0.367
	centroid	27.3	30.3	31.3	32.5	0.427	0.404	0.391	0.367
	class	27.3	30.2	31.3	32.4	0.427	0.403	0.390	0.366
Med-S2S	random	25.4	27.8	28.7	30.0	0.402	0.379	0.366	0.344
	centroid	25.3	27.6	28.8	30.1	0.403	0.380	0.366	0.345
	class	25.3	27.7	28.7	30.0	0.402	0.379	0.366	0.344
SE-S2S	random	31.7	55.5	57.9	59.2	0.710	0.690	0.675	0.644
	centroid	31.6	55.2	57.9	59.1	0.710	0.690	0.674	0.644
	class	31.6	55.3	57.9	59.2	0.710	0.690	0.674	0.644
20NG	random	32.3	39.3	41.3	43.3	0.718	0.670	0.636	0.571
	centroid	32.2	39.3	40.9	43.1	0.717	0.670	0.636	0.570
	class	32.0	39.2	41.3	43.3	0.718	0.670	0.636	0.570

The centroid and class orderings concentrate the seed of the incremental construction as described in the text; random is the ordering used elsewhere in the paper. Values are means over ten runs. The adversarial seeds leave both quality and structure essentially unchanged

Table 10 Descriptive statistics of the incremental graphs across the ten random node orderings: transitivity, average local clustering coefficient, degree assortativity, and label homophily at $k=1$ and $k=6$, averaged over dataset partitions

Dataset	$k=1$					$k=6$				
	trans	avg. clust	assort	homoph	trans	avg. clust	assort	homoph	trans	avg. clust
Arx-S2S	0.000 _{0,000}	0.000 _{0,000}	-0.082 _{0,008}	0.550 _{0,002}	0.134 _{0,001}	0.205 _{0,001}	0.084 _{0,005}	0.509 _{0,001}	0.134 _{0,001}	0.205 _{0,001}
Bio-S2S	0.000 _{0,000}	0.000 _{0,000}	-0.072 _{0,013}	0.440 _{0,004}	0.146 _{0,001}	0.220 _{0,002}	0.084 _{0,008}	0.392 _{0,002}	0.146 _{0,001}	0.220 _{0,002}
Med-S2S	0.000 _{0,000}	0.000 _{0,000}	-0.055 _{0,022}	0.402 _{0,006}	0.158 _{0,002}	0.263 _{0,003}	0.032 _{0,010}	0.344 _{0,003}	0.158 _{0,002}	0.263 _{0,003}
Red-S2S	0.000 _{0,000}	0.000 _{0,000}	-0.071 _{0,009}	0.672 _{0,002}	0.140 _{0,001}	0.212 _{0,001}	0.058 _{0,005}	0.612 _{0,001}	0.140 _{0,001}	0.212 _{0,001}
SE-S2S	0.000 _{0,000}	0.000 _{0,000}	-0.068 _{0,009}	0.710 _{0,003}	0.137 _{0,001}	0.199 _{0,001}	0.107 _{0,005}	0.644 _{0,001}	0.137 _{0,001}	0.199 _{0,001}
20NG	0.000 _{0,000}	0.000 _{0,000}	-0.194 _{0,009}	0.718 _{0,004}	0.249 _{0,002}	0.433 _{0,002}	0.068 _{0,006}	0.571 _{0,002}	0.249 _{0,002}	0.433 _{0,002}
Arx-P2P	0.000 _{0,000}	0.000 _{0,000}	-0.081 _{0,007}	0.628 _{0,002}	0.140 _{0,001}	0.213 _{0,001}	0.077 _{0,004}	0.589 _{0,001}	0.140 _{0,001}	0.213 _{0,001}
Bio-P2P	0.000 _{0,000}	0.000 _{0,000}	-0.072 _{0,012}	0.485 _{0,004}	0.165 _{0,001}	0.246 _{0,001}	0.102 _{0,008}	0.437 _{0,002}	0.165 _{0,001}	0.246 _{0,001}
Med-P2P	0.000 _{0,000}	0.000 _{0,000}	-0.066 _{0,018}	0.428 _{0,006}	0.167 _{0,002}	0.273 _{0,002}	0.053 _{0,009}	0.367 _{0,003}	0.167 _{0,002}	0.273 _{0,002}
Red-P2P	0.000 _{0,000}	0.000 _{0,000}	-0.069 _{0,011}	0.806 _{0,002}	0.143 _{0,001}	0.214 _{0,001}	0.066 _{0,005}	0.759 _{0,001}	0.143 _{0,001}	0.214 _{0,001}
SE-P2P	0.000 _{0,000}	0.000 _{0,000}	-0.083 _{0,012}	0.285 _{0,004}	0.133 _{0,001}	0.191 _{0,002}	0.099 _{0,006}	0.248 _{0,002}	0.133 _{0,001}	0.191 _{0,002}

Each cell reports the across-ordering mean with the across-ordering standard deviation as a subscript. Graph density and edge count are identical across orderings by construction and are omitted. At $k=1$ the incremental construction yields a spanning tree, hence zero transitivity and clustering

exactly k edges, so different orderings can change which edges appear but not how many, and the edge counts are indeed identical across orderings in all 1, 416 (dataset, partition, k) cells. The structural statistics are likewise stable: across orderings, transitivity, average local clustering, degree assortativity, and label homophily vary only in the third decimal (worst-case across-ordering standard deviations 0.005, 0.010, 0.032, and 0.012, respectively). A variance decomposition over all cells attributes less than 0.04% of the total variance of transitivity, average clustering, and homophily, and 0.9% of the variance of assortativity, to the node ordering; the remainder is explained by the dataset, partition, and k . The graphs produced under different orderings are therefore structurally almost identical, and the stability of these statistics provides no evidence that particular insertion orders select structurally favorable regions of the embedding space. The same invariance holds under the two adversarial orderings: as the right block of Table 9 shows, relative to the random ordering the centroid and class seeds change label homophily by at most 0.005 (mean $|\Delta| = 0.001$ over all 300 evaluated dataset–partition– k cells), leave the degree distribution unchanged, and always yield a single connected component, so the concentrated seeds do not place the construction in structurally more favorable regions either. Table 10 shows $k = 1$ and $k = 6$; the statistics for all evaluated k are provided in Appendix B.

7 Conclusion and Further Work

The main contribution of this work is an incremental k -NN graph construction algorithm that guarantees connectedness of the neighborhood graph, together with its theoretical and empirical analysis. In contrast to related approaches, the proposed algorithm achieves connectedness using only the search for nearest neighbours without requiring additional information to ensure the graph's connectivity.

As shown in Sect. 4, the neighborhood graph created with Algorithm 1 will always be connected. We show (in Sect. 3) that k -NN neighborhood graphs inferred on realistic datasets can exhibit disconnected components for commonly used values of k , and in some cases up to $k = 20$ (see Sect. 6). Furthermore, when compared to standard k -NN, clustering performance is significantly improved for low values of k and comparable for higher values of k . Although the incremental nature of the Algorithm 1 makes it sensitive to the ordering of the nodes, we show that the variance of the resulting graphs has little impact on the clustering performance. In Sect. 6.2.1, we show that larger embedding models for the initial high-dimensional representation consistently improve the results of our approach. Unlike approaches described in Sect. 2 that rely on a minimum spanning tree to repair connectivity, the presented incremental approach guarantees connectedness by construction and requires no global structure: an added MST improves clustering only in the sparsest $k = 1$ regime and is redundant for $k \geq 3$ (Sect. 6.2.2). Under matched MST augmentation, our construction still matches or exceeds the standard k -NN graph at low k , while avoiding the quadratic memory cost of the exact MST computation (Sect. 6.2.4). Two limitations of the present evaluation deserve mention: the five largest RedditClusteringP2P partitions exceed the 32-bit index range of the sparse eigensolver and are excluded from the recomputed MST configurations (Tables 7 and 13), and the robustness study of Sect. 6.2.5 covers concentrated seeds but not fully temporally ordered streams; a 64-bit eigensolver and a timestamped corpus would remove both restrictions.

Proposed incremental k -NN neighborhood graph construction is simple and effective, but it may be suboptimal in its initial steps. Nodes that are processed at the start of the algorithm have a higher probability of connecting to a large number of weakly related nodes and disproportionately influence the information encoded with a graph. Future work can analyze alternative strategies to approach the early steps of the algorithm and reduce the risk of suboptimal performance. The neighborhood graph produced by the proposed algorithm can be efficiently extended and allows both the addition of new and the deletion of the last added nodes. Those properties were not used in the clustering evaluation as we only evaluated the final clustering performance. Future work can exploit those properties by focusing on applications where new data is continuously added (streaming data) or invalidated. We have shown that in order to get clustering performance comparable to spectral clustering on the standard k -NN neighborhood graph, the nearest-neighbor relation does not need to be exact. Future work can explore using approximate k -NN search, e.g., using the algorithm described in Malkov and Yashunin (2020).

One of the open problems in graph-based approaches to clustering is the recalculation of all intermediate steps when new data points are introduced (Mondal et al., 2024). With the incremental construction of the neighborhood graph, we make an initial step in addressing this issue, as the existing neighborhood graph can be reused and easily extended with new data. This property can be exploited in combination with backpropagation-based eigen decomposition (Wang et al., 2019), or a variant of efficient update of eigenvectors (Dhanjal et al., 2014). Future work should apply those approaches and directly quantify the benefits of an incremental approach to data with a temporal dimension. Such evaluation was done for temporal community detection in Sattar et al. (2023), where the authors underline the necessity for new scalable graph algorithms for temporal data, a setting in which the proposed construction could serve as a useful graph-building component.

Appendix A: Graph Statistics Used

- Graph density measures how close the graph is to being a complete graph. It is defined as:

$$\text{density}(G) = \frac{2|E|}{|V|(|V| - 1)}$$

- Assortativity refers to the tendency of nodes in the graph to be connected to other nodes that are similar in some way. In degree assortativity, it is quantified by the Pearson correlation coefficient of the degrees between pairs of connected nodes:

$$r = \frac{L^{-1} \sum_i j_i k_i - [L^{-1} \sum_i \frac{1}{2}(j_i + k_i)]^2}{L^{-1} \sum_i \frac{1}{2}(j_i^2 + k_i^2) - [L^{-1} \sum_i \frac{1}{2}(j_i + k_i)]^2}$$

where j_i and k_i are the degrees of the nodes at the ends of the i th edge, and L is the total number of edges.

- Transitivity measures the overall degree of clustering in the graph, defined as:

$$C = \frac{3 \times \text{number of triangles in the network}}{\text{number of connected triangles of the nodes}}$$

- Local clustering coefficient measures the degree to which nodes in a graph tend to cluster together. For a given node v it is defined as

$$C_v = \frac{2|E_v|}{k_v(k_v - 1)},$$

where $|E_v|$ is the number of edges between the neighbors of v , and k_v is the degree of v .

- Pagerank is a measure of node importance based on the structure of the incoming links and is defined for a node as follows:

$$PR(i) = \frac{1-d}{N} + d \sum_{j \in M(i)} \frac{PR(j)}{L(j)},$$

where N is the total number of nodes, $M(i)$ is the set of nodes connected to i , $L(j)$ is the number of outgoing connections to node j , and d is the damping factor, which is usually set to 0.85.

- Homophily is a tendency of individual nodes to connect and link with similar other nodes. It measures the similarity of connected nodes with respect to a specific attribute, in our case, the true clustering label. For a graph $G = (V, E)$ where each node (document) $v \in V$ has a *cluster label*, the homophily H of the graph can be quantified as the proportion of edges that connect nodes with the same label:

$$H = \frac{|\{(v_i, v_j) \in E : \text{label}(v_i) = \text{label}(v_j)\}|}{|E|},$$

where $\text{label}(v)$ denotes the label of the node v and $|\{(v_i, v_j) \in E : \text{label}(v_i) = \text{label}(v_j)\}|$ is the number of edges where both documents have the same label.

Appendix B: Graph Statistics Across Node Orderings for all k

Tables 11 and 12 extend Table 10 to all evaluated neighborhood sizes $k \in \{1, 2, 3, 6, 8, 10, 15, 20\}$, reporting transitivity, the average local clustering coefficient, degree assortativity, and label homophily of the incremental graphs (across-ordering mean over the ten random orderings with the across-ordering standard deviation as a subscript, averaged over dataset partitions). Graph density and edge count are identical across orderings by construction and are omitted. At $k = 1$ the incremental construction yields a spanning tree, hence zero transitivity and clustering at every dataset. The across-ordering variability remains in the third decimal for every dataset and k , consistent with the variance decomposition reported in Sect. 6.2.5.

Table 11 Graph statistics of the incremental construction across the ten random node orderings for all evaluated k_i sentence-level (S2S) tasks: across-ordering mean with the standard deviation as a subscript, averaged over dataset partitions

Dataset	k_i	transitivity	avg. clustering	assortativity	homophily
Arx-S2S	1	0.000 _{0,000}	0.000 _{0,000}	-0.082 _{0,008}	0.550 _{0,002}
	2	0.080 _{0,001}	0.183 _{0,002}	0.000 _{0,006}	0.536 _{0,002}
	3	0.107 _{0,001}	0.194 _{0,001}	0.036 _{0,005}	0.527 _{0,002}
	6	0.134 _{0,001}	0.205 _{0,001}	0.084 _{0,005}	0.509 _{0,001}
	8	0.141 _{0,001}	0.208 _{0,001}	0.099 _{0,004}	0.501 _{0,001}
	10	0.146 _{0,001}	0.210 _{0,001}	0.110 _{0,004}	0.495 _{0,001}
	15	0.154 _{0,000}	0.213 _{0,001}	0.127 _{0,004}	0.483 _{0,001}
	20	0.159 _{0,000}	0.215 _{0,001}	0.137 _{0,004}	0.474 _{0,001}
Bio-S2S	1	0.000 _{0,000}	0.000 _{0,000}	-0.072 _{0,013}	0.440 _{0,004}
	2	0.088 _{0,002}	0.200 _{0,004}	0.012 _{0,009}	0.421 _{0,003}
	3	0.116 _{0,001}	0.209 _{0,002}	0.044 _{0,007}	0.410 _{0,003}
	6	0.146 _{0,001}	0.220 _{0,002}	0.084 _{0,008}	0.392 _{0,002}
	8	0.155 _{0,001}	0.224 _{0,001}	0.096 _{0,007}	0.384 _{0,002}
	10	0.161 _{0,001}	0.227 _{0,001}	0.103 _{0,007}	0.378 _{0,002}
	15	0.172 _{0,001}	0.232 _{0,001}	0.114 _{0,007}	0.367 _{0,002}
	20	0.179 _{0,001}	0.237 _{0,001}	0.119 _{0,006}	0.359 _{0,002}
Med-S2S	1	0.000 _{0,000}	0.000 _{0,000}	-0.055 _{0,022}	0.402 _{0,006}
	2	0.096 _{0,003}	0.241 _{0,006}	0.000 _{0,014}	0.379 _{0,005}
	3	0.125 _{0,002}	0.250 _{0,004}	0.017 _{0,011}	0.366 _{0,004}
	6	0.158 _{0,002}	0.263 _{0,003}	0.032 _{0,010}	0.344 _{0,003}
	8	0.168 _{0,002}	0.268 _{0,002}	0.036 _{0,009}	0.335 _{0,003}
	10	0.176 _{0,002}	0.272 _{0,002}	0.038 _{0,009}	0.328 _{0,003}
	15	0.191 _{0,002}	0.280 _{0,002}	0.041 _{0,008}	0.315 _{0,003}
	20	0.202 _{0,002}	0.287 _{0,002}	0.042 _{0,008}	0.305 _{0,003}

Table 11 (continued)

Dataset	k	transitivity	avg. clustering	assortativity	homophily
Red-S2S	1	0.000 _{0,000}	0.000 _{0,000}	-0.071 _{0,009}	0.672 _{0,002}
	2	0.084 _{0,001}	0.193 _{0,003}	-0.001 _{0,006}	0.652 _{0,002}
	3	0.111 _{0,001}	0.202 _{0,002}	0.026 _{0,005}	0.638 _{0,002}
	6	0.140 _{0,001}	0.212 _{0,001}	0.058 _{0,005}	0.612 _{0,001}
	8	0.149 _{0,001}	0.216 _{0,001}	0.069 _{0,004}	0.600 _{0,001}
	10	0.155 _{0,001}	0.219 _{0,001}	0.076 _{0,004}	0.590 _{0,001}
	15	0.166 _{0,001}	0.224 _{0,001}	0.087 _{0,004}	0.570 _{0,001}
	20	0.174 _{0,001}	0.229 _{0,001}	0.095 _{0,003}	0.555 _{0,001}
SE-S2S	1	0.000 _{0,000}	0.000 _{0,000}	-0.068 _{0,009}	0.710 _{0,003}
	2	0.082 _{0,001}	0.175 _{0,003}	0.024 _{0,006}	0.690 _{0,002}
	3	0.108 _{0,001}	0.186 _{0,002}	0.062 _{0,006}	0.675 _{0,002}
	6	0.137 _{0,001}	0.199 _{0,001}	0.107 _{0,005}	0.644 _{0,001}
	8	0.145 _{0,001}	0.203 _{0,001}	0.121 _{0,005}	0.630 _{0,001}
	10	0.151 _{0,001}	0.205 _{0,001}	0.131 _{0,005}	0.618 _{0,001}
	15	0.160 _{0,001}	0.210 _{0,001}	0.145 _{0,005}	0.595 _{0,001}
	20	0.166 _{0,001}	0.213 _{0,001}	0.154 _{0,004}	0.577 _{0,001}
20NG	1	0.000 _{0,000}	0.000 _{0,000}	-0.194 _{0,009}	0.718 _{0,004}
	2	0.156 _{0,002}	0.502 _{0,004}	-0.107 _{0,008}	0.670 _{0,003}
	3	0.208 _{0,002}	0.479 _{0,003}	-0.038 _{0,008}	0.636 _{0,003}
	6	0.249 _{0,002}	0.433 _{0,002}	0.068 _{0,006}	0.571 _{0,002}
	8	0.254 _{0,002}	0.412 _{0,002}	0.101 _{0,007}	0.543 _{0,002}
	10	0.255 _{0,001}	0.396 _{0,002}	0.119 _{0,006}	0.521 _{0,002}
	15	0.254 _{0,001}	0.370 _{0,002}	0.139 _{0,006}	0.481 _{0,002}
	20	0.252 _{0,001}	0.354 _{0,001}	0.146 _{0,005}	0.453 _{0,002}

Table 12 Graph statistics of the incremental construction across the ten random node orderings for all evaluated k , paragraph-level (P2P) tasks: across-ordering mean with the standard deviation as a subscript, averaged over dataset partitions

Dataset	k	Transitivity	avg. clustering	Assortativity	Homophily
Arx-P2P	1	0.000 _{0.000}	0.000 _{0.000}	-0.081 _{0.007}	0.628 _{0.002}
	2	0.081 _{0.001}	0.183 _{0.002}	0.000 _{0.005}	0.615 _{0.002}
	3	0.109 _{0.001}	0.196 _{0.001}	0.034 _{0.005}	0.606 _{0.002}
	6	0.140 _{0.001}	0.213 _{0.001}	0.077 _{0.004}	0.589 _{0.001}
	8	0.150 _{0.001}	0.219 _{0.001}	0.091 _{0.004}	0.582 _{0.001}
	10	0.156 _{0.001}	0.223 _{0.001}	0.101 _{0.004}	0.576 _{0.001}
	15	0.167 _{0.000}	0.230 _{0.001}	0.116 _{0.003}	0.564 _{0.001}
	20	0.174 _{0.000}	0.235 _{0.001}	0.125 _{0.003}	0.555 _{0.001}
Bio-P2P	1	0.000 _{0.000}	0.000 _{0.000}	-0.072 _{0.012}	0.485 _{0.004}
	2	0.101 _{0.002}	0.224 _{0.005}	0.020 _{0.008}	0.467 _{0.003}
	3	0.132 _{0.001}	0.234 _{0.003}	0.057 _{0.008}	0.456 _{0.003}
	6	0.165 _{0.001}	0.246 _{0.001}	0.102 _{0.008}	0.437 _{0.002}
	8	0.175 _{0.001}	0.250 _{0.001}	0.115 _{0.007}	0.429 _{0.002}
	10	0.182 _{0.001}	0.253 _{0.001}	0.124 _{0.006}	0.422 _{0.002}
	15	0.193 _{0.001}	0.259 _{0.001}	0.136 _{0.006}	0.410 _{0.002}
	20	0.201 _{0.001}	0.264 _{0.001}	0.142 _{0.005}	0.401 _{0.002}
Med-P2P	1	0.000 _{0.000}	0.000 _{0.000}	-0.066 _{0.018}	0.428 _{0.006}
	2	0.102 _{0.002}	0.250 _{0.005}	0.003 _{0.012}	0.404 _{0.005}
	3	0.133 _{0.003}	0.260 _{0.004}	0.028 _{0.011}	0.390 _{0.004}
	6	0.167 _{0.002}	0.273 _{0.002}	0.053 _{0.009}	0.367 _{0.003}
	8	0.178 _{0.002}	0.277 _{0.002}	0.060 _{0.009}	0.358 _{0.003}
	10	0.186 _{0.002}	0.282 _{0.002}	0.063 _{0.008}	0.351 _{0.003}
	15	0.199 _{0.002}	0.289 _{0.002}	0.067 _{0.007}	0.337 _{0.003}
	20	0.209 _{0.002}	0.296 _{0.002}	0.068 _{0.007}	0.328 _{0.003}
Red-P2P	1	0.000 _{0.000}	0.000 _{0.000}	-0.069 _{0.011}	0.806 _{0.002}
	2	0.079 _{0.001}	0.176 _{0.002}	0.004 _{0.007}	0.792 _{0.001}
	3	0.108 _{0.001}	0.192 _{0.002}	0.033 _{0.006}	0.781 _{0.001}
	6	0.143 _{0.001}	0.214 _{0.001}	0.066 _{0.005}	0.759 _{0.001}
	8	0.154 _{0.001}	0.223 _{0.001}	0.077 _{0.005}	0.748 _{0.001}
	10	0.162 _{0.001}	0.228 _{0.001}	0.084 _{0.005}	0.739 _{0.001}
	15	0.177 _{0.001}	0.239 _{0.001}	0.095 _{0.005}	0.721 _{0.001}
	20	0.186 _{0.001}	0.247 _{0.001}	0.101 _{0.004}	0.707 _{0.001}
SE-P2P	1	0.000 _{0.000}	0.000 _{0.000}	-0.083 _{0.012}	0.285 _{0.004}
	2	0.075 _{0.001}	0.157 _{0.004}	0.011 _{0.008}	0.272 _{0.003}
	3	0.102 _{0.001}	0.172 _{0.002}	0.050 _{0.007}	0.264 _{0.002}
	6	0.133 _{0.001}	0.191 _{0.002}	0.099 _{0.006}	0.248 _{0.002}
	8	0.143 _{0.001}	0.198 _{0.001}	0.114 _{0.006}	0.241 _{0.002}
	10	0.150 _{0.001}	0.203 _{0.001}	0.125 _{0.006}	0.236 _{0.002}
	15	0.162 _{0.001}	0.211 _{0.001}	0.141 _{0.005}	0.226 _{0.002}
	20	0.170 _{0.001}	0.217 _{0.001}	0.151 _{0.005}	0.219 _{0.001}

Appendix C: Bayesian Practical-Equivalence Analysis of the MST Contribution

Table 13 reports the Bayesian signed-rank comparison of the incremental graph with and without MST augmentation discussed in Sect. 6.2.2. Table 14 extends the analysis to every evaluated k , reporting the posterior probability of practical equivalence; the probability that the un-augmented graph is practically better never exceeds 0.011 in any cell, so the complementary mass always lies on the Ours+MST side. Equivalence is reached on most dataset variants already at $k = 2$ –3 and on every variant from $k = 6$ on.

Table 13 Bayesian signed-rank assessment (ROPE = 0.01) of the incremental graph with and without MST augmentation, run separately at $k = 1$ and $k = 6$

Dataset	$k = 1$			$k = 6$		
	P(Ours)	P(ROPE)	P(+MST)	P(Ours)	P(ROPE)	P(+MST)
ArxivClusteringP2P	0.000	0.000	1.000	0.000	1.000	0.000
ArxivClusteringS2S	0.000	0.000	1.000	0.000	1.000	0.000
BiorxivClusteringP2P	0.000	0.000	1.000	0.000	1.000	0.000
BiorxivClusteringS2S	0.000	0.000	1.000	0.000	1.000	0.000
MedrxivClusteringP2P	0.000	0.000	1.000	0.000	1.000	0.000
MedrxivClusteringS2S	0.000	0.000	1.000	0.000	1.000	0.000
RedditClustering	0.000	0.000	1.000	0.000	1.000	0.000
RedditClusteringP2P	0.000	0.001	0.999	0.000	1.000	0.000
StackExchangeClustering	0.000	0.000	1.000	0.000	1.000	0.000
StackExchangeClusteringP2P	0.000	1.000	0.000	0.000	1.000	0.000
TwentyNewsgroupsClustering	0.000	0.000	1.000	0.000	1.000	0.000

Each test uses one paired sample per partition (the V-measure mean over the ten node orderings), so its samples are independent; the RedditClusteringP2P samples exclude the five largest partitions, as in Table 7. Bold marks the dominant posterior outcome

Table 14 Posterior probability of practical equivalence, $P(\text{ROPE})$ with ROPE = 0.01, between Ours and Ours+MST for every evaluated k , computed as in Table 13 with one paired sample per partition. Bold marks cells where equivalence is the dominant outcome

Dataset	$k=1$	$k=2$	$k=3$	$k=6$	$k=8$	$k=10$	$k=15$	$k=20$
ArxivClusteringP2P	0.000	0.993	1.000	1.000	1.000	1.000	1.000	1.000
ArxivClusteringS2S	0.000	0.998	1.000	1.000	1.000	1.000	1.000	1.000
BiorxivClusteringP2P	0.000	0.993	1.000	1.000	1.000	1.000	1.000	1.000
BiorxivClusteringS2S	0.000	0.955	1.000	1.000	1.000	1.000	1.000	1.000
MedrxivClusteringP2P	0.000	0.106	1.000	1.000	1.000	1.000	1.000	1.000
MedrxivClusteringS2S	0.000	0.045	1.000	1.000	1.000	1.000	1.000	1.000
RedditClustering	0.000	0.000	0.518	1.000	1.000	1.000	1.000	1.000
RedditClusteringP2P	0.001	0.218	0.767	1.000	1.000	1.000	0.996	1.000
StackExchangeClustering	0.000	0.000	1.000	1.000	1.000	1.000	1.000	1.000
StackExchangeClusteringP2P	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
TwentyNewsgroupsClustering	0.000	0.818	0.989	1.000	1.000	0.991	1.000	0.981

The complementary probability mass lies on the Ours+MST side in every cell: $P(\text{Ours better}) \leq 0.011$ throughout

Acknowledgements The work was partially supported by the Slovenian Research and Innovation Agency (ARIS) core research programmes P2-0103 and P6-0411, as well as the projects EMMA (L2-50070), and LLM4DH (GC-0002). The work of Boshko Koloski was supported by a young researcher grant PR-12394. The work was also supported by the EU through ERA Chair grant no. 101186647 (AI4DH).

Author Contributions Authorship contributions. All authors contributed to the conceptualization of the study. Marko Pranjic? and Boshko Koloski contributed equally and led the experiments and analysis. Marko Pranjic? and Boshko Koloski drafted the manuscript and carried out the writing, review, and editing. Nada Lavra?, Senja Pollak, and Marko Robnik-Šikonja contributed to interpretation of results and manuscript revisions.

Data Availability No datasets were generated or analysed during the current study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. In J. Bussche & V. Vianu (Eds.), *Database Theory – ICDT 2001* (pp. 420–434). Berlin, Heidelberg: Springer.
- Aggarwal, C. C., & Zhai, C. (2012). *A survey of text clustering algorithms. Mining text data* (pp. 77–128).
- Alexopoulos, C., & Fishman, G. S. (1991). Characterizing stochastic flow networks using the monte carlo method. *Networks*, 21(7), 775–798.
- Angelov, D. (2020). Top2vec: Distributed representations of topics [arXiv:2008.09470](https://arxiv.org/abs/2008.09470) arXiv preprint.
- Avrachenkov, K., Litvak, N., Nemirovsky, D., & Osipova, N. (2007). Monte carlo methods in pagerank computation: When one iteration is sufficient. *SIAM Journal on Numerical Analysis*, 45(2), 890–904.
- Beguerisse-Díaz, M., Vangelov, B., & Barahona, M. (2013). Finding role communities in directed networks using role-based similarity, markov stability and the relaxed minimum spanning tree. *2013 IEEE Global Conference on Signal and Information Processing* (pp. 937–940).
- Belkin, M., & Niyogi, P. (2003). Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6), 1373–1396.
- Benavoli, A., Corani, G., Demšar, J., & Zaffalon, M. (2017). Time for a change: a tutorial for comparing multiple classifiers through bayesian analysis. *Journal of Machine Learning Research*, 18(77), 1–36.
- Berry, T., & Sauer, T. (2019). Consistent manifold representation for topological data analysis. *Foundations of Data Science*, 1(1), 1–38.
- Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When is “nearest neighbor” meaningful? In C. Beeri & P. Buneman (Eds.), *Database Theory – ICDT’99* (pp. 217–235). Berlin, Heidelberg: Springer.
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008, 10008. (IOP Publishing)
- Campello, R. J., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Pacific-Asia Conference on Knowledge Discovery and Data Mining* (pp. 160–172). Springer.
- Corizzo, R., Pio, G., Barracchia, E. P., Pellicani, A., Japkowicz, N., & Ceci, M. (2023). HURI: Hybrid user risk identification in social networks. *World Wide Web*. <https://doi.org/10.1007/s11280-023-01192-w>
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *The Journal of Machine learning research*, 7, 1–30.
- Derron-Pinion, A., She, J., Wong, D., Lange, O., Hester, T., Perez, L., Nunkesser, M., Lee, S., Guo, X., Wiltshire, B., et al. (2021) Eta prediction with graph neural networks in google maps. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pp. 3767–3776
- Dhanjal, C., Gaudel, R., & Cléménçon, S. (2014). Efficient eigen-updating for spectral graph clustering. *Neurocomputing*, 131, 440–452. <https://doi.org/10.1016/j.neucom.2013.11.015>

- Dhillon, I. S., & Modha, D. S. (2001). Concept decompositions for large sparse text data using clustering. *Machine Learning*, 42(1), 143–175.
- Dong, J., Chen, C., Zhang, C., Ma, J., & Peng, K. (2025). Knowledge graph embedding with graph convolutional network and bidirectional gated recurrent unit for fault diagnosis of industrial processes. *IEEE Sensors Journal*. <https://doi.org/10.1109/JSEN.2025.3528223>
- Ester, M., Kriegl, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)* (pp. 226–231). AAAI Press.
- Grootendorst, M. (2022). Bertopic: Neural topic modeling with a class-based tf-idf procedure [arXiv:2203.05794](https://arxiv.org/abs/2203.05794) arXiv preprint.
- Jarvis, R. A., & Patrick, E. A. (1973). Clustering using a similarity measure based on shared near neighbors. *IEEE Transactions on Computers C*, 22(11), 1025–1034. <https://doi.org/10.1109/T-C.1973.223640>
- Jha, K., Saha, S., & Singh, H. (2022). Prediction of protein-protein interaction using graph neural networks. *Scientific Reports*, 12(1), 8360.
- Koloski, B., Pranjic, M., Lavrac, N., Skrlj, B., & Pollak, S. (2023). Inducing document representations from graphs: A blueprint. In: Maughan, K., Liu, R., Burns, T.F. (eds.) *The First Tiny Papers Track at ICLR 2023, Tiny Papers @ ICLR 2023*, Kigali, Rwanda, May 5, 2023. OpenReview.net, Kigali, Rwanda. <https://openreview.net/pdf?id=2rp3guEM3A>
- Li, Z., Zhang, X., Zhang, Y., Long, D., Xie, P., & Zhang, M. (2023). Towards general text embeddings with multi-stage contrastive learning. arXiv preprint [arXiv:2308.03281](https://arxiv.org/abs/2308.03281)
- Liu, Z., & Barahona, M. (2020). Graph-based data clustering via multiscale community detection. *Applied Network Science*, 5.
- Luxburg, U. (2007). A tutorial on spectral clustering. *Statistics and Computing*, 17(4), 395–416. <https://doi.org/10.1007/s11222-007-9033-z>
- MacQueen, J. (1967) Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297. Oakland, CA, USA
- Maier, M., Hein, M., & Luxburg, U. (2009). Optimal construction of k-nearest-neighbor graphs for identifying noisy clusters. *Theoretical Computer Science*, 410(19), 1749–1764. <https://doi.org/10.1016/j.tcs.2009.01.009>
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.
- Margeloiu, A., Simidjievski, N., Lio, P., & Jamnik, M. (2024). GCondnet: A novel method for improving neural networks on small high-dimensional tabular data. *Transactions on Machine Learning Research, Reproducibility Certification*.
- McInnes, L., Healy, J., & Astels, S. (2017) hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software* 2(11)
- McInnes, L., Healy, J., Saul, N., & Großberger, L. (2018) UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software* 3(29), 861. <https://doi.org/10.21105/joss.00861>
- Mondal, R., Ignatova, E., Walke, D., Broneske, D., Saake, G., & Heyer, R. (2024). Clustering graph data: the roadmap to spectral techniques. *Discover Artificial Intelligence*, 4(1), 7. <https://doi.org/10.1007/s44163-024-00102-x>
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive text embedding benchmark. *Proc. of the 17th Conference of the European Chapter of the Association for Computational Linguistics* (pp. 2014–2037). ACL, Dubrovnik, Croatia
- Murtagh, F., & Contreras, P. (2012). Algorithms for hierarchical clustering: an overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 2(1), 86–97.
- Ng, A. Y., Jordan, M. I., & Weiss, Y. (2001). On spectral clustering: Analysis and an algorithm. *Proc. of the 14th International Conference on Neural Information Processing Systems: Natural and Synthetic. NIPS'01* (pp. 849–856). Cambridge, MA, USA: MIT Press.
- Radovanović, M., Nanopoulos, A., & Ivanović, M. (2010). Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11, 2487–2531.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proc. of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982–3992). Hong Kong, China: Association for Computational Linguistics.
- Reiser, P., Neubert, M., Eberhard, A., Torresi, L., Zhou, C., Shao, C., Metni, H., Hoesel, C., Schopmans, H., Sommer, T., et al. (2022). Graph neural networks for materials science and chemistry. *Communications Materials*, 3(1), 93.

- Rosenberg, A., & Hirschberg, J. (2007). V-measure: A conditional entropy-based external cluster evaluation measure. *Proc. of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)* (pp. 410–420). Prague, Czech Republic: Association for Computational Linguistics.
- Sattar, N. S., Buluc, A., Ibrahim, K. Z., & Arifuzzaman, S. (2023). Exploring temporal community evolution: algorithmic approaches and parallel optimization for dynamic community detection. *Applied Network Science*, 8(1), 64. <https://doi.org/10.1007/s41109-023-00592-1>
- Seyerlehner, K., Flexer, A., & Widmer, G. (2009). On the limitations of browsing top-n recommender systems. *Proceedings of the Third ACM Conference on Recommender Systems. RecSys '09* (pp. 321–324). New York, NY, USA: Association for Computing Machinery.
- Škrlj, B., Kralj, J., & Lavrač, N. (2020). Embedding-based silhouette community detection. *Machine Learning*, 109, 2161–2193.
- Spielman, D. A., & Srivastava, N. (2008) Graph sparsification by effective resistances. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing (STOC)*, pp. 563–568. <https://doi.org/10.1145/1374376.1374456>
- Tenenbaum, J. B., Silva, V., & Langford, J. C. (2000). A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500), 2319–2323.
- Toussaint, G. T. (1980). The relative neighbourhood graph of a finite planar set. *Pattern Recognition*, 12(4), 261–268. [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7)
- Traag, V. A., Waltman, L., & Van Eck, N. J. (2019). From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1), 5233.
- Tsitsulin, A., Palowitch, J., Perozzi, B., & Müller, E. (2023). Graph clustering with graph neural networks. *Journal of Machine Learning Research*, 24(127), 1–21.
- Veenstra, P., Cooper, C., & Phelps, S. (2016) Spectral clustering using the knn-mst similarity graph, pp. 222–227. <https://doi.org/10.1109/CEEC.2016.7835917>
- Wang, X., Bo, D., Shi, C., Fan, S., Ye, Y., & Yu, P. S. (2023). A survey on heterogeneous graph embedding: Methods, techniques, applications and sources. *IEEE Transactions on Big Data*. <https://doi.org/10.1109/TBDDATA.2022.3177455>
- Wang, W., Dang, Z., Hu, Y., Fua, P., & Salzmann, M. (2019). *Backpropagation-friendly eigendecomposition*. Red Hook, NY, USA: Curran Associates Inc.
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-pack: Packed resources for general chinese embeddings [arXiv:2309.07597](https://arxiv.org/abs/2309.07597) arXiv preprint.
- Xie, J., Girshick, R., & Farhadi, A. (2016). Unsupervised deep embedding for clustering analysis. *Proceedings of the 33rd International Conference on Machine Learning* (pp. 478–487) PMLR.
- Xue, F., & Kumar, P. R. (2004). The number of neighbors needed for connectivity of wireless networks. *Wirel. Netw.*, 10(2), 169–181. <https://doi.org/10.1023/B:WINE.0000013081.09837.c0>
- Zaripova, K., Cosmo, L., Kazi, A., Ahmadi, S.-A., Bronstein, M. M., & Navab, N. (2023). Graph-in-graph (gig): Learning interpretable latent graphs in non-euclidean domain for biological and healthcare applications. *Medical Image Analysis*, 88, Article 102839.
- Zelnik-Manor, L., & Perona, P. (2004). Self-tuning spectral clustering. *Advances in Neural Information Processing Systems (NIPS)* (Vol. 17,
- Zemel, R., & Carreira-Perpiñán, M. (2004). Proximity graphs for clustering and manifold learning. In L. Saul, Y. Weiss, & L. Bottou (Eds.), *Advances in Neural Information Processing Systems*. (Vol. 17). Cambridge, MA: MIT Press.
- Zhang, P., Ren, Y., & Zhang, B. (2012). A new embedding quality assessment method for manifold learning. *Neurocomputing*, 97, 251–266.